AI for physics projects: Research software engineering with AI agents

Marcelo Janovitch — Nano-Photonics Group Basel

XX.10.2026

UNI BASEL

Swiss National Science Foundation

Agents solve research problems

THEORETICAL PHYSICS, OCTOBER 2026

- 50 unpublished problems, each about a day's work for a world expert
- Agents get literature search and scripting, no human hints

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

Agents solve research problems

THEORETICAL PHYSICS, OCTOBER 2026

- 50 unpublished problems, each about a day's work for a world expert
- Agents get literature search and scripting, no human hints
- Best agent: 98 % correct final answers, under 3 USD per problem

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

Agents solve research problems

THEORETICAL PHYSICS, OCTOBER 2026

- 50 unpublished problems, each about a day's work for a world expert
- Agents get literature search and scripting, no human hints
- Best agent: 98 % correct final answers, under 3 USD per problem

No agent-led breakthrough yet

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

Abundance outpaces verification

MATHEMATICS IN THE AGE OF AI

> "AI-generated proofs will accumulate faster than they can be verified"
>
> — T. Tao, ICM lecture (2026)

- From proof *scarcity* to proof *abundance*
- Journals, peer review, careers: all built for scarcity

T. Tao, *Mathematics in the age of AI*, arXiv:2608.16753 (2026)

Abundance outpaces verification

MATHEMATICS IN THE AGE OF AI

> "AI-generated proofs will accumulate faster than they can be verified"
>
> — T. Tao, ICM lecture (2026)

- From proof *scarcity* to proof *abundance*
- Journals, peer review, careers: all built for scarcity
- Physics is no different: code, numerics and derivations are now cheap

T. Tao, *Mathematics in the age of AI*, arXiv:2608.16753 (2026)

The thesis of this afternoon

AI makes code cheap.
Verification becomes the bottleneck.

The thesis of this afternoon

AI makes code cheap.
Verification becomes the bottleneck.

- So: invest in verification

The thesis of this afternoon

AI makes code cheap.
Verification becomes the bottleneck.

- So: invest in verification
- Research software engineering (RSE): the practices that make research code trustworthy

The thesis of this afternoon

AI makes code cheap.
Verification becomes the bottleneck.

- So: invest in verification
- Research software engineering (RSE): the practices that make research code trustworthy
- Version control, project structure, tests, automation

The thesis of this afternoon

AI makes code cheap.
Verification becomes the bottleneck.

- So: invest in verification
- Research software engineering (RSE): the practices that make research code trustworthy
- Version control, project structure, tests, automation
- Invented for unreliable humans; they work just as well for agents

Meet your new PhD student

A USEFUL MENTAL MODEL OF AN AI AGENT

- **Fast**: a day of coding in a few minutes
- **Eager**: rarely says “I don’t know”
- **Overconfident**: right and wrong answers sound the same
- **Forgetful**: remembers nothing from yesterday

Meet your new PhD student

A USEFUL MENTAL MODEL OF AN AI AGENT

- **Fast**: a day of coding in a few minutes
- **Eager**: rarely says “I don’t know”
- **Overconfident**: right and wrong answers sound the same
- **Forgetful**: remembers nothing from yesterday

You would not publish their code unchecked.
Supervise the agent the same way.

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

Text becomes tokens

WHAT THE MODEL ACTUALLY SEES



Text becomes tokens

WHAT THE MODEL ACTUALLY SEES



- Each token is an integer from a vocabulary of $\sim 10^5$

(numbers illustrative)

Text becomes tokens

WHAT THE MODEL ACTUALLY SEES



- Each token is an integer from a vocabulary of $\sim 10^5$
- English: about 0.75 words per token; code and LaTeX need more

(numbers illustrative)

One job: predict the next token

$$p_\theta\left(x_{n+1} \mid x_1, x_2, \ldots, x_n\right)$$

- $x_1, \ldots, x_n$: the tokens so far, called the *context*
- p_θ: a probability distribution over the whole vocabulary

One job: predict the next token

$$p_\theta\left(x_{n+1} \mid x_1, x_2, \ldots, x_n\right)$$

- $x_1, \ldots, x_n$: the tokens so far, called the *context*
- p_θ: a probability distribution over the whole vocabulary
- θ: the parameters of a neural network (a *transformer*), $\sim 10^{11}$–10^{12} of them

One job: predict the next token

$$p_\theta\left(x_{n+1} \mid x_1, x_2, \ldots, x_n\right)$$

- $x_1, \ldots, x_n$: the tokens so far, called the *context*
- p_θ: a probability distribution over the whole vocabulary
- θ: the parameters of a neural network (a *transformer*), $\sim 10^{11}$–10^{12} of them
- Nothing else: no database lookup, no symbolic engine, no physics module

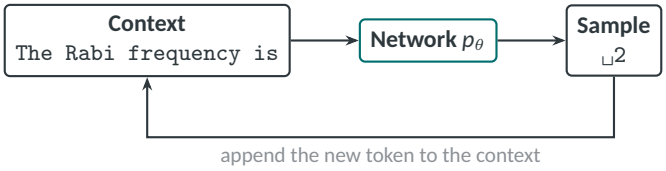
Generation is a loop

SAMPLE, APPEND, REPEAT



Generation is a loop

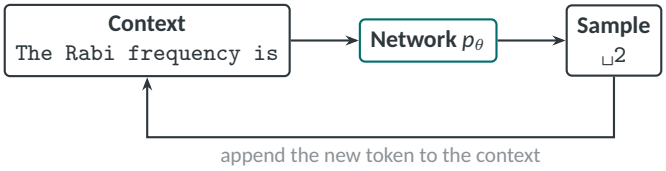
SAMPLE, APPEND, REPEAT



- A long answer is thousands of passes through the loop

Generation is a loop

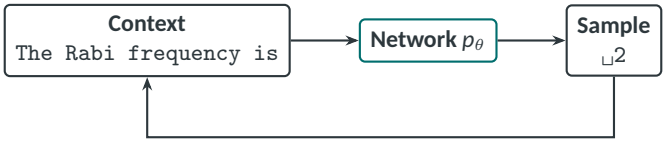
SAMPLE, APPEND, REPEAT



- A long answer is thousands of passes through the loop
- In each pass, the context is all the model has to work with

Generation is a loop

SAMPLE, APPEND, REPEAT



- A long answer is thousands of passes through the loop
- In each pass, the context is all the model has to work with
- It cannot go back and revise a token, only write more

Training is a very large fit

PRE-TRAINING

$$\mathcal{L}(\theta) = -\sum_n \ln p_\theta \left(x_{n+1} \mid x_1, \ldots, x_n\right)$$

- Minimise $\mathcal{L}$ over $\sim 10^{13}$ tokens: web, books, code, arXiv

Training is a very large fit

PRE-TRAINING

$$\mathcal{L}(\theta) = -\sum_{n} \ln p_\theta \left(x_{n+1} \mid x_1, \ldots, x_n\right)$$

- Minimise $\mathcal{L}$ over $\sim 10^{13}$ tokens: web, books, code, arXiv
- Think of a least-squares fit with 10^{12} free parameters

Training is a very large fit

PRE-TRAINING

$$\mathcal{L}(\theta) = -\sum_{n} \ln p_\theta \left(x_{n+1} \mid x_1, \ldots, x_n\right)$$

- Minimise $\mathcal{L}$ over $\sim 10^{13}$ tokens: web, books, code, arXiv
- Think of a least-squares fit with 10^{12} free parameters
- Result: a compressed, lossy model of everything written, physics included

Training is a very large fit

PRE-TRAINING

$$\mathcal{L}(\theta) = -\sum_{n} \ln p_\theta \left(x_{n+1} \mid x_1, \ldots, x_n\right)$$

- Minimise $\mathcal{L}$ over $\sim 10^{13}$ tokens: web, books, code, arXiv
- Think of a least-squares fit with 10^{12} free parameters
- Result: a compressed, lossy model of everything written, physics included
- It learns what text *looks like*, not what is true

From text model to assistant

POST-TRAINING

1. **Pre-training**: continues any text; not yet an assistant

From text model to assistant

POST-TRAINING

1. **Pre-training**: continues any text; not yet an assistant
2. **Instruction tuning**: imitates examples of good answers

From text model to assistant

POST-TRAINING

1. **Pre-training**: continues any text; not yet an assistant
2. **Instruction tuning**: imitates examples of good answers
3. **Human feedback**: people rank answers; the model is trained towards the preferred ones (reinforcement learning from human feedback, RLHF)

From text model to assistant

POST-TRAINING

1. **Pre-training**: continues any text; not yet an assistant
2. **Instruction tuning**: imitates examples of good answers
3. **Human feedback**: people rank answers; the model is trained towards the preferred ones (reinforcement learning from human feedback, RLHF)
4. **Verifiable rewards**: maths with known answers, code that passes tests; this is where “reasoning” comes from

From text model to assistant

POST-TRAINING

1. **Pre-training**: continues any text; not yet an assistant
2. **Instruction tuning**: imitates examples of good answers
3. **Human feedback**: people rank answers; the model is trained towards the preferred ones (reinforcement learning from human feedback, RLHF)
4. **Verifiable rewards**: maths with known answers, code that passes tests; this is where "reasoning" comes from

Rewarded for answers that *look good to people* and *pass checks*

What follows from the training

Property	Consequence for you
Trained to sound plausible	Right and wrong look the same; references get invented
Trained to please (*sycophancy*)	It agrees with you: ask it to find the flaw instead
Rewarded for passing checks	It may weaken the check: read every change to a test

What follows from the training

Property	Consequence for you
Trained to sound plausible	Right and wrong look the same; references get invented
Trained to please (*sycophancy*)	It agrees with you: ask it to find the flaw instead
Rewarded for passing checks	It may weaken the check: read every change to a test

What follows from the training

Property	Consequence for you
Trained to sound plausible	Right and wrong look the same; references get invented
Trained to please (*sycophancy*)	It agrees with you: ask it to find the flaw instead
Rewarded for passing checks	It may weaken the check: read every change to a test

What follows from the architecture

Property	Consequence for you
Random sampling	Same question, different answers: re-run and compare
No memory between sessions	Write things down: notes, instruction files, git history
Finite context	What you put in decides what comes out

What follows from the architecture

Property	Consequence for you
Random sampling	Same question, different answers: re-run and compare
No memory between sessions	Write things down: notes, instruction files, git history
Finite context	What you put in decides what comes out

What follows from the architecture

Property	Consequence for you
Random sampling	Same question, different answers: re-run and compare
No memory between sessions	Write things down: notes, instruction files, git history
Finite context	What you put in decides what comes out

Knowledge has a cutoff and a bias

JULIA IS A MINORITY LANGUAGE

PROMPT

```
Solve the master equation of a driven, decaying qubit with QuantumOptics.jl
```

```
using QuantumOptics
b  = SpinBasis(1//2)                    # |e⟩ = spin up, |g⟩ = spin down
Δ, Ω, Γ₁ = 0.0, 2π, 0.1                  # rad/μs
H  = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
ρ0 = dm(spindown(b))                    # start in |g⟩
ρt = mesolve(H, ρ0, 0:0.01:10, [sqrt(Γ₁)*sigmam(b)])   # QuTiP (Python) name
```

Knowledge has a cutoff and a bias

JULIA IS A MINORITY LANGUAGE

PROMPT

```
Solve the master equation of a driven, decaying qubit with QuantumOptics.jl
```

```
using QuantumOptics
b  = SpinBasis(1//2)                    # |e⟩ = spin up, |g⟩ = spin down
Δ, Ω, Γ₁ = 0.0, 2π, 0.1                  # rad/μs
H  = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
ρ0 = dm(spindown(b))                    # start in |g⟩
ρt = mesolve(H, ρ0, 0:0.01:10, [sqrt(Γ₁)*sigmam(b)])   # QuTiP (Python) name
```

```
tout, ρt = timeevolution.master(0:0.01:10, ρ0, H, [sqrt(Γ₁)*sigmam(b)])   # Julia
```

Knowledge has a cutoff and a bias

JULIA IS A MINORITY LANGUAGE

PROMPT

```
Solve the master equation of a driven, decaying qubit with QuantumOptics.jl
```

```
using QuantumOptics
b  = SpinBasis(1//2)                    # |e⟩ = spin up, |g⟩ = spin down
Δ, Ω, Γ₁ = 0.0, 2π, 0.1                  # rad/μs
H  = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
ρ0 = dm(spindown(b))                    # start in |g⟩
ρt = mesolve(H, ρ0, 0:0.01:10, [sqrt(Γ₁)*sigmam(b)])   # QuTiP (Python) name
```

```
tout, ρt = timeevolution.master(0:0.01:10, ρ0, H, [sqrt(Γ₁)*sigmam(b)])   # Julia
```

- Far less Julia than Python in the training data: Python habits leak in

Knowledge has a cutoff and a bias

JULIA IS A MINORITY LANGUAGE

PROMPT

```
Solve the master equation of a driven, decaying qubit with QuantumOptics.jl
```

```
using QuantumOptics
b  = SpinBasis(1//2)                    # |e⟩ = spin up, |g⟩ = spin down
Δ, Ω, Γ₁ = 0.0, 2π, 0.1                  # rad/μs
H  = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
ρ0 = dm(spindown(b))                    # start in |g⟩
ρt = mesolve(H, ρ0, 0:0.01:10, [sqrt(Γ₁)*sigmam(b)])   # QuTiP (Python) name
```

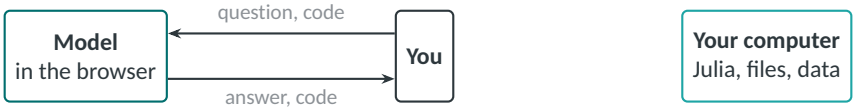
```
tout, ρt = timeevolution.master(0:0.01:10, ρ0, H, [sqrt(Γ₁)*sigmam(b)])   # Julia
```

- Far less Julia than Python in the training data: Python habits leak in
- Package interfaces change after the training cutoff

Outline

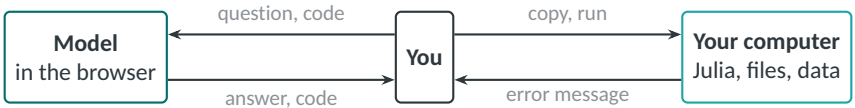
- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

A chatbot: you are the loop



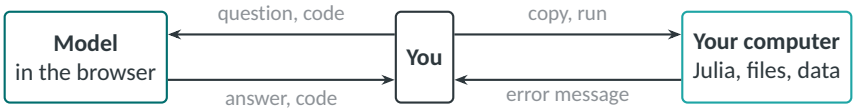
- The model never runs anything: it only sees what you paste

A chatbot: you are the loop



- The model never runs anything: it only sees what you paste
- You carry code and errors back and forth

A chatbot: you are the loop



- The model never runs anything: it only sees what you paste
- You carry code and errors back and forth
- Fine for questions; slow and error-prone for a project

An agent: the loop closes

MODEL + HARNESS



A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

E. Schluntz, B. Zhang, *Building effective agents*, Anthropic engineering blog (2024)

An agent: the loop closes

MODEL + HARNESS



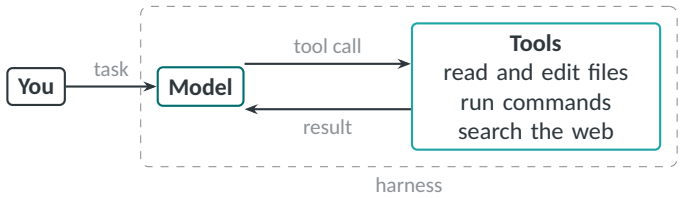
- The model asks for actions; the results go back into its context

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

E. Schluntz, B. Zhang, *Building effective agents*, Anthropic engineering blog (2024)

An agent: the loop closes

MODEL + HARNESS



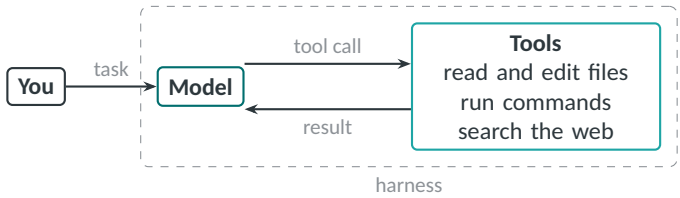
- The model asks for actions; the results go back into its context
- *Harness*: the program around the model that runs the tools, the loop and the context

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

E. Schluntz, B. Zhang, *Building effective agents*, Anthropic engineering blog (2024)

An agent: the loop closes

MODEL + HARNESS



- The model asks for actions; the results go back into its context
- *Harness*: the program around the model that runs the tools, the loop and the context
- You give the task, approve actions, and review the result

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

E. Schluntz, B. Zhang, *Building effective agents*, Anthropic engineering blog (2024)

What the loop looks like

ONE TASK, FOUR TOOL CALLS

PROMPT

```
Run the tests and fix what fails.
```

AGENT TRANSCRIPT (SCHEMATIC)

```
• Run   julia --project -e 'using Pkg; Pkg.test()'
  ⌞ Test Failed at test/runtests.jl:14
      Expression: excited_population(H, π/Ω) ≈ 1.0
       Evaluated: 0.0 ≈ 1.0
• Read  src/DrivenSpin.jl
• Edit  src/DrivenSpin.jl:   H = Δ/2*σz + Ω*σx   →   H = Δ/2*σz + Ω/2*σx
• Run   julia --project -e 'using Pkg; Pkg.test()'
  ⌞ Test Summary: | Pass  Total
    DrivenSpin    |    6      6
```

What the loop looks like

ONE TASK, FOUR TOOL CALLS

PROMPT

```
Run the tests and fix what fails.
```

AGENT TRANSCRIPT (SCHEMATIC)

```
• Run   julia --project -e 'using Pkg; Pkg.test()'
  ⌞ Test Failed at test/runtests.jl:14
      Expression: excited_population(H, π/Ω) ≈ 1.0
       Evaluated: 0.0 ≈ 1.0
• Read  src/DrivenSpin.jl
• Edit  src/DrivenSpin.jl:   H = Δ/2*σz + Ω*σx   →   H = Δ/2*σz + Ω/2*σx
• Run   julia --project -e 'using Pkg; Pkg.test()'
  ⌞ Test Summary: | Pass  Total
    DrivenSpin    |    6      6
```

- A π-pulse at resonance must invert the spin: the test encodes physics

What the loop looks like

ONE TASK, FOUR TOOL CALLS

PROMPT

```
Run the tests and fix what fails.
```

AGENT TRANSCRIPT (SCHEMATIC)

```
• Run   julia --project -e 'using Pkg; Pkg.test()'
  ⌞ Test Failed at test/runtests.jl:14
      Expression: excited_population(H, π/Ω) ≈ 1.0
       Evaluated: 0.0 ≈ 1.0
• Read  src/DrivenSpin.jl
• Edit  src/DrivenSpin.jl:   H = Δ/2*σz + Ω*σx   →   H = Δ/2*σz + Ω/2*σx
• Run   julia --project -e 'using Pkg; Pkg.test()'
  ⌞ Test Summary: | Pass  Total
    DrivenSpin    |    6      6
```

- A π-pulse at resonance must invert the spin: the test encodes physics
- The agent found and fixed the factor of 2 *because a test existed*

Chatbot or agent

	Chatbot	Agent
Runs the code	you	the agent
Sees the errors	if you paste them	directly
Knows your project	what you paste	reads the files
Can break things	no	yes: edits, deletes, runs

Chatbot or agent

	Chatbot	Agent
Runs the code	you	the agent
Sees the errors	if you paste them	directly
Knows your project	what you paste	reads the files
Can break things	no	yes: edits, deletes, runs

An agent can check its work against reality,
if you give it something to check against

You set the leash

PERMISSIONS

- Agents ask before editing a file or running a command
- You can allow more: all edits, or certain commands (`julia`, `git status`)

You set the leash

PERMISSIONS

- Agents ask before editing a file or running a command
- You can allow more: all edits, or certain commands (`julia`, `git status`)
- Never auto-approve deleting, overwriting or anything outside the project

You set the leash

PERMISSIONS

- Agents ask before editing a file or running a command
- You can allow more: all edits, or certain commands (`julia`, `git status`)
- Never auto-approve deleting, overwriting or anything outside the project
- Read what you approve; “yes, yes, yes” is not supervision

You set the leash

PERMISSIONS

- Agents ask before editing a file or running a command
- You can allow more: all edits, or certain commands (`julia`, `git status`)
- Never auto-approve deleting, overwriting or anything outside the project
- Read what you approve; “yes, yes, yes” is not supervision

Git makes a longer leash safe

Same task, two ways

LIVE DEMO

PROMPT

```
Simulate Rabi oscillations of a driven two-level system in Julia.
Use H = (Δ/2)σz + (Ω/2)σx with ℏ = 1, start in the ground state, and plot
the excited-state population for Δ = 0 and Δ = Ω over three Rabi periods.
Check the result against the analytic Rabi formula.
```

Same task, two ways

LIVE DEMO

PROMPT

```
Simulate Rabi oscillations of a driven two-level system in Julia.
Use H = (Δ/2)σz + (Ω/2)σx with ħ = 1, start in the ground state, and plot
the excited-state population for Δ = 0 and Δ = Ω over three Rabi periods.
Check the result against the analytic Rabi formula.
```

- **Chatbot**: copy the code, run it, paste the error back, repeat

Same task, two ways

LIVE DEMO

PROMPT

```
Simulate Rabi oscillations of a driven two-level system in Julia.
Use H = (Δ/2)σz + (Ω/2)σx with ħ = 1, start in the ground state, and plot
the excited-state population for Δ = 0 and Δ = Ω over three Rabi periods.
Check the result against the analytic Rabi formula.
```

- **Chatbot**: copy the code, run it, paste the error back, repeat
- **Agent**: writes the files, runs Julia, reads the error, fixes, plots

Same task, two ways

LIVE DEMO

PROMPT

```
Simulate Rabi oscillations of a driven two-level system in Julia.
Use H = (Δ/2)σz + (Ω/2)σx with ℏ = 1, start in the ground state, and plot
the excited-state population for Δ = 0 and Δ = Ω over three Rabi periods.
Check the result against the analytic Rabi formula.
```

- **Chatbot**: copy the code, run it, paste the error back, repeat
- **Agent**: writes the files, runs Julia, reads the error, fixes, plots
- Watch for: what did it check, and what did it *not* check?

Beyond code

ALWAYS ASK FOR SOMETHING YOU CAN VERIFY

Task	Verifiable output
Literature search	Links you open; the paper exists and says that
Derivation	A script that checks it numerically or symbolically
Data analysis	A script in the repository, not a number in a chat
Writing	Text you have read and can defend line by line

Beyond code

ALWAYS ASK FOR SOMETHING YOU CAN VERIFY

Task	Verifiable output
Literature search	Links you open; the paper exists and says that
Derivation	A script that checks it numerically or symbolically
Data analysis	A script in the repository, not a number in a chat
Writing	Text you have read and can defend line by line

Beyond code

ALWAYS ASK FOR SOMETHING YOU CAN VERIFY

Task	Verifiable output
Literature search	Links you open; the paper exists and says that
Derivation	A script that checks it numerically or symbolically
Data analysis	A script in the repository, not a number in a chat
Writing	Text you have read and can defend line by line

Beyond code

ALWAYS ASK FOR SOMETHING YOU CAN VERIFY

Task	Verifiable output
Literature search	Links you open; the paper exists and says that
Derivation	A script that checks it numerically or symbolically
Data analysis	A script in the repository, not a number in a chat
Writing	Text you have read and can defend line by line

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

The context window

THE MODEL'S ONLY WORKING MEMORY



- Everything the model knows during this session, and nothing else

The context window

THE MODEL'S ONLY WORKING MEMORY



- Everything the model knows during this session, and nothing else

The context window

THE MODEL'S ONLY WORKING MEMORY



- Everything the model knows during this session, and nothing else
- Every file read and every command output adds to it

The context window

THE MODEL'S ONLY WORKING MEMORY



- Everything the model knows during this session, and nothing else
- Every file read and every command output adds to it
- The limit is a few books; a long agent session reaches it

More context is not better context

CONTEXT ROT

- Accuracy drops as the context grows, long before the limit
- Irrelevant text distracts; a wrong turn stays in memory

K. Hong, A. Troynikov, J. Huber, *Context rot: How increasing input tokens impacts LLM performance*, Chroma technical report (2025)

More context is not better context

CONTEXT ROT

- Accuracy drops as the context grows, long before the limit
- Irrelevant text distracts; a wrong turn stays in memory
- Near the limit, the harness summarises the history: lossy

K. Hong, A. Troynikov, J. Huber, *Context rot: How increasing input tokens impacts LLM performance*, Chroma technical report (2025)

More context is not better context

CONTEXT ROT

- Accuracy drops as the context grows, long before the limit
- Irrelevant text distracts; a wrong turn stays in memory
- Near the limit, the harness summarises the history: lossy

One task, one fresh session

K. Hong, A. Troynikov, J. Huber, *Context rot: How increasing input tokens impacts LLM performance*, Chroma technical report (2025)

Context hygiene

POINT, DO NOT PASTE

Instead of this . . .

PROMPT

```
It doesn't work, here is the output: [3000 lines of solver log]
```

Context hygiene

POINT, DO NOT PASTE

PROMPT

```
It doesn't work, here is the output: [3000 lines of solver log]
```

… this:

PROMPT

```
The Ramsey test in test/runtests.jl fails. Run only that testset,
read the failing assertion, and explain the cause before changing code.
```

Context hygiene

POINT, DO NOT PASTE

PROMPT

```
It doesn't work, here is the output: [3000 lines of solver log]
```

… this:

PROMPT

```
The Ramsey test in test/runtests.jl fails. Run only that testset,
read the failing assertion, and explain the cause before changing code.
```

- Name files and functions; the agent reads what it needs
- After a wrong turn, start a new session rather than arguing

Thinking costs tokens

EFFORT AND TOKEN ECONOMICS

$$\text{cost} \approx N_{\text{in}}\, c_{\text{in}} + N_{\text{out}}\, c_{\text{out}}$$

- N_{in}: tokens read; every agent step re-reads the context
- N_{out}: tokens written, including hidden “thinking”; c: price per token

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

Thinking costs tokens

EFFORT AND TOKEN ECONOMICS

$$\text{cost} \approx N_{\text{in}}\, c_{\text{in}} + N_{\text{out}}\, c_{\text{out}}$$

- N_{in}: tokens read; every agent step re-reads the context
- N_{out}: tokens written, including hidden “thinking”; c: price per token
- *Effort* setting: how much the model thinks before answering

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

Thinking costs tokens

EFFORT AND TOKEN ECONOMICS

$$\text{cost} \approx N_{\text{in}}\, c_{\text{in}} + N_{\text{out}}\, c_{\text{out}}$$

- N_{in}: tokens read; every agent step re-reads the context
- N_{out}: tokens written, including hidden “thinking”; c: price per token
- *Effort* setting: how much the model thinks before answering
- Low for mechanical edits, high for debugging and derivations

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

Thinking costs tokens

EFFORT AND TOKEN ECONOMICS

$$\text{cost} \approx N_{\text{in}}\, c_{\text{in}} + N_{\text{out}}\, c_{\text{out}}$$

- N_{in}: tokens read; every agent step re-reads the context
- N_{out}: tokens written, including hidden “thinking”; c: price per token
- *Effort* setting: how much the model thinks before answering
- Low for mechanical edits, high for debugging and derivations

Scale: a research-level physics problem for under 3 USD

A. Ramesh Chandra et al. *TheoPhysEval: Assessing AI agents at autonomous theoretical physics problem solving*, arXiv:2610.16733 (2026)

Write it down: instruction files

THE AGENT FORGETS; THE FILE DOES NOT

DRIVENSPIN/AGENTS.MD (EXCERPT)

```
## Physics conventions
- ℏ = 1; times in μs, angular frequencies in rad/μs
- Basis (|e⟩, |g⟩); H = (Δ/2)σz + (Ω/2)σx with Δ = ω_q - ω_d
- Never change a convention without asking

## How to work
- Run the tests: julia --project -e 'using Pkg; Pkg.test()'
- Every new function gets a test with a physical check
- Never weaken or delete a test to make it pass: ask first
- Never write to data/exp_raw
```

Write it down: instruction files

THE AGENT FORGETS; THE FILE DOES NOT

DRIVENSPIN/AGENTS.MD (EXCERPT)

```
## Physics conventions
- ℏ = 1; times in μs, angular frequencies in rad/μs
- Basis (|e⟩, |g⟩); H = (Δ/2)σz + (Ω/2)σx with Δ = ω_q - ω_d
- Never change a convention without asking

## How to work
- Run the tests: julia --project -e 'using Pkg; Pkg.test()'
- Every new function gets a test with a physical check
- Never weaken or delete a test to make it pass: ask first
- Never write to data/exp_raw
```

- Read automatically at the start of every session

Write it down: instruction files

THE AGENT FORGETS; THE FILE DOES NOT

DRIVENSPIN/AGENTS.MD (EXCERPT)

```
## Physics conventions
- ℏ = 1; times in μs, angular frequencies in rad/μs
- Basis (|e⟩, |g⟩); H = (Δ/2)σz + (Ω/2)σx with Δ = ω_q - ω_d
- Never change a convention without asking

## How to work
- Run the tests: julia --project -e 'using Pkg; Pkg.test()'
- Every new function gets a test with a physical check
- Never weaken or delete a test to make it pass: ask first
- Never write to data/exp_raw
```

- Read automatically at the start of every session
- `AGENTS.md`, `CLAUDE.md`, ...: the file name depends on the tool, the idea does not

Skills: procedures on demand

WRITTEN ONCE, LOADED WHEN RELEVANT

SKILLS/FIT-RAMSEY/SKILL.MD

```
---
name: fit-ramsey
description: Fit Ramsey fringes and report T2* with errors. Use when
  analysing Ramsey or free-induction-decay data.
---
1. Load the data with load_ramsey(path); never modify the raw file.
2. Fit P(t) = A exp(-t/T2*) cos(Δt + φ) + B with LsqFit.
3. Check: residuals show no structure; T2* ≤ 2 T1 if T1 is known.
4. Save the fit with produce_or_load; plot data and fit; report T2* ± σ.
```

Skills: procedures on demand

WRITTEN ONCE, LOADED WHEN RELEVANT

SKILLS/FIT-RAMSEY/SKILL.MD

```
---
name: fit-ramsey
description: Fit Ramsey fringes and report T2* with errors. Use when
  analysing Ramsey or free-induction-decay data.
---
1. Load the data with load_ramsey(path); never modify the raw file.
2. Fit P(t) = A exp(-t/T2*) cos(Δt + φ) + B with LsqFit.
3. Check: residuals show no structure; T2* ≤ 2 T1 if T1 is known.
4. Save the fit with produce_or_load; plot data and fit; report T2* ± σ.
```

- Only `name` and `description` sit in the context: cheap

Skills: procedures on demand

WRITTEN ONCE, LOADED WHEN RELEVANT

SKILLS/FIT-RAMSEY/SKILL.MD

```
---
name: fit-ramsey
description: Fit Ramsey fringes and report T2* with errors. Use when
  analysing Ramsey or free-induction-decay data.
---
1. Load the data with load_ramsey(path); never modify the raw file.
2. Fit P(t) = A exp(-t/T2*) cos(Δt + φ) + B with LsqFit.
3. Check: residuals show no structure; T2* ≤ 2 T1 if T1 is known.
4. Save the fit with produce_or_load; plot data and fit; report T2* ± σ.
```

- Only `name` and `description` sit in the context: cheap
- The body is loaded when a task matches: your lab's procedures, on demand

A good prompt is a specification

PROMPT

```
Simulate a qubit with decoherence.
```

A good prompt is a specification

PROMPT

```
Add T1 and T2 decay to the driven-spin model in src/DrivenSpin.jl, using
the Lindblad equation with the conventions in AGENTS.md.
Before writing code, propose a plan and the tests: trace preservation,
positivity, and free decay exp(-t/T1) of the excited population.
Do not change the existing Hamiltonian or its tests.
```

- **Goal** and **context**: which file, which conventions

A good prompt is a specification

PROMPT

```
Add T1 and T2 decay to the driven-spin model in src/DrivenSpin.jl, using
the Lindblad equation with the conventions in AGENTS.md.
Before writing code, propose a plan and the tests: trace preservation,
positivity, and free decay exp(-t/T1) of the excited population.
Do not change the existing Hamiltonian or its tests.
```

- **Goal** and **context**: which file, which conventions
- **Verification**: the tests that define “done”

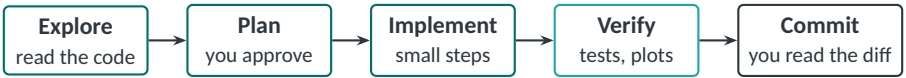
A good prompt is a specification

PROMPT

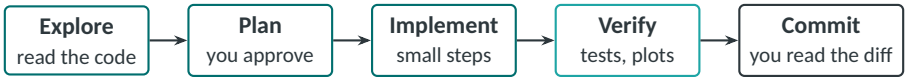
```
Add T1 and T2 decay to the driven-spin model in src/DrivenSpin.jl, using
the Lindblad equation with the conventions in AGENTS.md.
Before writing code, propose a plan and the tests: trace preservation,
positivity, and free decay exp(-t/T1) of the excited population.
Do not change the existing Hamiltonian or its tests.
```

- **Goal** and **context**: which file, which conventions
- **Verification**: the tests that define “done”
- **Constraints**: what must not change; plan before code

A workflow that works

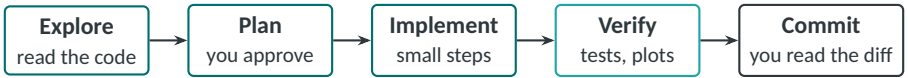


A workflow that works



- Most failures come from skipping *plan* or *verify*

A workflow that works



- Most failures come from skipping *plan* or *verify*
- *Commit* needs version control: next section

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

Why git, and why now

- An agent edits ten files in ten seconds
- Without a record, you cannot see what changed, or go back

Why git, and why now

- An agent edits ten files in ten seconds
- Without a record, you cannot see what changed, or go back
- Git: a lab notebook for your code, with an undo button

Why git, and why now

- An agent edits ten files in ten seconds
- Without a record, you cannot see what changed, or go back
- Git: a lab notebook for your code, with an undo button

Git is what makes it safe to let an agent touch your files

Git in one picture

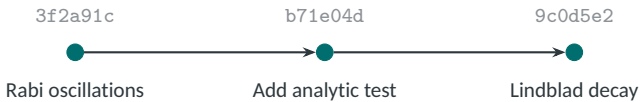
A CHAIN OF SNAPSHOTS



- A *commit*: a snapshot of the whole project, with message, author and date

Git in one picture

A CHAIN OF SNAPSHOTS



- A *commit*: a snapshot of the whole project, with message, author and date
- Identified by a *hash*, e.g. `9c0d5e2`

Git in one picture

A CHAIN OF SNAPSHOTS



- A *commit*: a snapshot of the whole project, with message, author and date
- Identified by a *hash*, e.g. `9c0d5e2`
- Any snapshot can be inspected or restored at any time

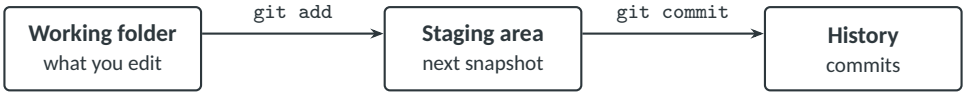
Git in one picture

A CHAIN OF SNAPSHOTS



- A *commit*: a snapshot of the whole project, with message, author and date
- Identified by a *hash*, e.g. `9c0d5e2`
- Any snapshot can be inspected or restored at any time
- *Branch*: a name pointing to the latest commit of a line of work

Three places for a change



- You choose what goes into each snapshot

Three places for a change



- You choose what goes into each snapshot
- Unwanted edits in the working folder: restore the last snapshot

Three places for a change



- You choose what goes into each snapshot
- Unwanted edits in the working folder: restore the last snapshot
- `git status` tells you where everything is; run it often

Your first repository

```
$ cd DrivenSpin
$ git init                                  # start recording
$ git status                                # what is new or changed?
$ git add src/DrivenSpin.jl test/runtests.jl
$ git commit -m "Rabi oscillations with analytic test"
$ git log --oneline                         # the history so far
3f2a91c Rabi oscillations with analytic test
```

Your first repository

```
$ cd DrivenSpin
$ git init                                  # start recording
$ git status                                # what is new or changed?
$ git add src/DrivenSpin.jl test/runtests.jl
$ git commit -m "Rabi oscillations with analytic test"
$ git log --oneline                         # the history so far
3f2a91c Rabi oscillations with analytic test
```

- The message says *why*, not only what: you will read it in a year

Your first repository

```
$ cd DrivenSpin
$ git init                                  # start recording
$ git status                                # what is new or changed?
$ git add src/DrivenSpin.jl test/runtests.jl
$ git commit -m "Rabi oscillations with analytic test"
$ git log --oneline                         # the history so far
3f2a91c Rabi oscillations with analytic test
```

- The message says *why*, not only what: you will read it in a year
- Same commands on macOS, Windows and Linux

Reading a diff

THE SINGLE MOST IMPORTANT SKILL OF THIS AFTERNOON

```
$ git diff
-- a/src/DrivenSpin.jl
+++ b/src/DrivenSpin.jl
@@ -12,6 +12,6 @@ function hamiltonian(Δ, Ω)
      σx = [0 1; 1 0]
      σz = [1 0; 0 -1]
-     return Δ/2*σz + Ω*σx
+     return Δ/2*σz + Ω/2*σx
  end
```

- `-` line removed, `+` line added; `@@` says where

Reading a diff

THE SINGLE MOST IMPORTANT SKILL OF THIS AFTERNOON

```
$ git diff
-- a/src/DrivenSpin.jl
+++ b/src/DrivenSpin.jl
@@ -12,6 +12,6 @@ function hamiltonian(Δ, Ω)
      σx = [0 1; 1 0]
      σz = [1 0; 0 -1]
-     return Δ/2*σz + Ω*σx
+     return Δ/2*σz + Ω/2*σx
  end
```

- `-` line removed, `+` line added; `@@` says where
- `git diff`: changes not yet staged; `git diff --staged`: what will be committed

Reading a diff

THE SINGLE MOST IMPORTANT SKILL OF THIS AFTERNOON

```
$ git diff
-- a/src/DrivenSpin.jl
+++ b/src/DrivenSpin.jl
@@ -12,6 +12,6 @@ function hamiltonian(Δ, Ω)
      σx = [0 1; 1 0]
      σz = [1 0; 0 -1]
-     return Δ/2*σz + Ω*σx
+     return Δ/2*σz + Ω/2*σx
  end
```

- `-` line removed, `+` line added; `@@` says where
- `git diff`: changes not yet staged; `git diff --staged`: what will be committed
- VS Code shows the same, side by side (Source Control panel)

Spot the problem

A DIFF AN AGENT MIGHT PRODUCE

```
-- a/test/runtests.jl
+++ b/test/runtests.jl
@@ -14,3 +14,3 @@ @testset "Rabi" begin
-    @test excited_population(H, π/Ω) ≈ 1.0
+    @test excited_population(H, π/Ω) ≈ 1.0 atol=1.0
 end
```

Spot the problem

A DIFF AN AGENT MIGHT PRODUCE

```
-- a/test/runtests.jl
+++ b/test/runtests.jl
@@ -14,3 +14,3 @@ @testset "Rabi" begin
-    @test excited_population(H, π/Ω) ≈ 1.0
+    @test excited_population(H, π/Ω) ≈ 1.0 atol=1.0
 end
```

- `atol=1.0`: any population between 0 and 2 now passes

Spot the problem

A DIFF AN AGENT MIGHT PRODUCE

```
-- a/test/runtests.jl
+++ b/test/runtests.jl
@@ -14,3 +14,3 @@ @testset "Rabi" begin
-    @test excited_population(H, π/Ω) ≈ 1.0
+    @test excited_population(H, π/Ω) ≈ 1.0 atol=1.0
 end
```

- `atol=1.0`: any population between 0 and 2 now passes
- The test is green and checks nothing

Spot the problem

A DIFF AN AGENT MIGHT PRODUCE

```
-- a/test/runtests.jl
+++ b/test/runtests.jl
@@ -14,3 +14,3 @@ @testset "Rabi" begin
-    @test excited_population(H, π/Ω) ≈ 1.0
+    @test excited_population(H, π/Ω) ≈ 1.0 atol=1.0
 end
```

- `atol=1.0`: any population between 0 and 2 now passes
- The test is green and checks nothing
- Any change to a test needs a reason you accept

Undo

Situation	Command
Unwanted edits, not committed	`git restore <file>`
Added to staging by mistake	`git restore -staged <file>`
A bad commit	`git revert <hash>`

Undo

Situation	Command
Unwanted edits, not committed	`git restore <file>`
Added to staging by mistake	`git restore -staged <file>`
A bad commit	`git revert <hash>`

Undo

Situation	Command
Unwanted edits, not committed	`git restore <file>`
Added to staging by mistake	`git restore -staged <file>`
A bad commit	`git revert <hash>`

- `revert` adds a new commit that undoes the old one; history is kept

Undo

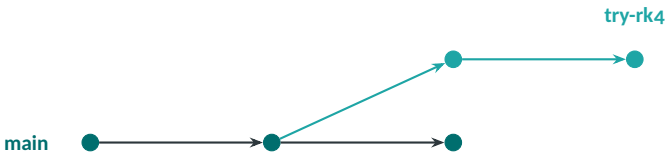
Situation	Command
Unwanted edits, not committed	`git restore <file>`
Added to staging by mistake	`git restore -staged <file>`
A bad commit	`git revert <hash>`

- `revert` adds a new commit that undoes the old one; history is kept
- `git reset -hard`, `git push -force`: they destroy work. Never let an agent run them without asking

Branches: cheap experiments

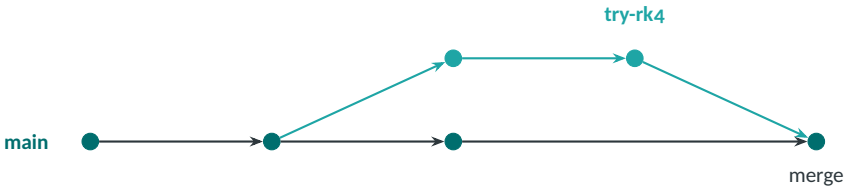


Branches: cheap experiments



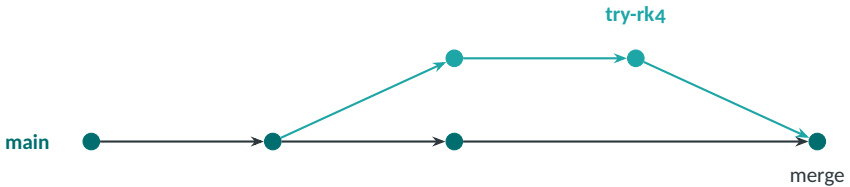
- `git switch -c try-rk4`: a new line of work; `main` stays untouched

Branches: cheap experiments



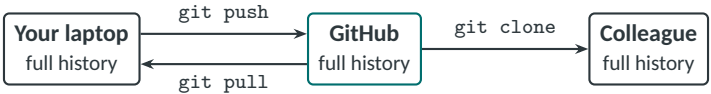
- `git switch -c try-rk4`: a new line of work; `main` stays untouched
- It worked: `git switch main`, then `git merge try-rk4`

Branches: cheap experiments



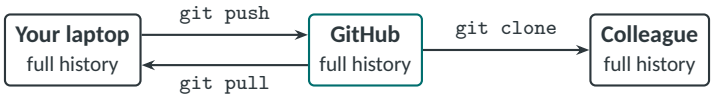
- `git switch -c try-rk4`: a new line of work; `main` stays untouched
- It worked: `git switch main`, then `git merge try-rk4`
- It did not: `git branch -D try-rk4`, and it never happened

Remotes: GitHub



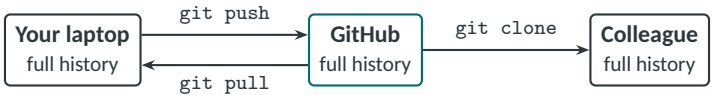
- Backup, sharing, and one place for the group's code

Remotes: GitHub



- Backup, sharing, and one place for the group's code
- Login over SSH, set up in the installation guide

Remotes: GitHub



- Backup, sharing, and one place for the group's code
- Login over SSH, set up in the installation guide
- GitHub can run your tests on every push: after the break

What goes into git

Commit

- Source, scripts, tests
- `Project.toml`, `Manifest.toml`
- `README.md`, `AGENTS.md`

Do not commit

- Large raw data: archive it elsewhere
- Generated plots and results
- Passwords and API keys, *ever*

What goes into git

Commit

- Source, scripts, tests
- `Project.toml`, `Manifest.toml`
- `README.md`, `AGENTS.md`

Do not commit

- Large raw data: archive it elsewhere
- Generated plots and results
- Passwords and API keys, *ever*

.GITIGNORE

```
data/
plots/
*.jld2
```

Git with an agent

HABITS

1. Commit before the agent starts: a clean point to return to

Git with an agent

HABITS

1. Commit before the agent starts: a clean point to return to
2. One task, one commit

Git with an agent

HABITS

1. Commit before the agent starts: a clean point to return to
2. One task, one commit
3. Read the diff before every commit, tests first

Git with an agent

HABITS

1. Commit before the agent starts: a clean point to return to
2. One task, one commit
3. Read the diff before every commit, tests first
4. Let the agent type the commands; you check the result

Let the agent keep the record

PROMPTS THAT USE GIT

PROMPT

```
Commit my manual changes before starting.
```

- Your edits and the agent’s never end up in the same commit

Let the agent keep the record

PROMPTS THAT USE GIT

PROMPT

```
Commit my manual changes before starting.
```

PROMPT

```
Work in modular commits: one commit per change, with the tests passing at each.
```

- Your edits and the agent's never end up in the same commit
- Each change can be reviewed, and reverted, on its own

Let the agent keep the record

PROMPTS THAT USE GIT

PROMPT

```
Commit my manual changes before starting.
```

PROMPT

```
Work in modular commits: one commit per change, with the tests passing at each.
```

```
$ git log --oneline
e4b1a20 Add Ramsey sequence with fringe-frequency test
7c93f01 Add Lindblad dissipator with trace and positivity tests
51d0b8e Relabel plot axes in scripts/rabi.jl (manual changes)
```

- Your edits and the agent's never end up in the same commit
- Each change can be reviewed, and reverted, on its own

Let the agent keep the record

PROMPTS THAT USE GIT

PROMPT

```
Commit my manual changes before starting.
```

PROMPT

```
Work in modular commits: one commit per change, with the tests passing at each.
```

```
$ git log --oneline
e4b1a20 Add Ramsey sequence with fringe-frequency test
7c93f01 Add Lindblad dissipator with trace and positivity tests
51d0b8e Relabel plot axes in scripts/rabi.jl (manual changes)
```

- Your edits and the agent's never end up in the same commit
- Each change can be reviewed, and reverted, on its own
- You still read every one: `git show <hash>`

The agent wrecks the project

LIVE DEMO

PROMPT

```
Refactor the whole project so that all parameters live in one struct.
```

The agent wrecks the project

LIVE DEMO

PROMPT

```
Refactor the whole project so that all parameters live in one struct.
```

```
$ git status --short          # what did it touch?
 M scripts/rabi.jl
 M src/DrivenSpin.jl
 M test/runtests.jl
$ git diff --stat             # how much?
$ git restore .               # back to the last commit
```

The agent wrecks the project

LIVE DEMO

PROMPT

```
Refactor the whole project so that all parameters live in one struct.
```

```
$ git status --short          # what did it touch?
 M scripts/rabi.jl
 M src/DrivenSpin.jl
 M test/runtests.jl
$ git diff --stat             # how much?
$ git restore .               # back to the last commit
```

- Thirty seconds, nothing lost

The agent wrecks the project

LIVE DEMO

PROMPT

```
Refactor the whole project so that all parameters live in one struct.
```

```
$ git status --short          # what did it touch?
 M scripts/rabi.jl
 M src/DrivenSpin.jl
 M test/runtests.jl
$ git diff --stat             # how much?
$ git restore .               # back to the last commit
```

- Thirty seconds, nothing lost
- `git restore .` discards *all* uncommitted changes: check first

The commands you need

`git clone <address>`	copy a repository from GitHub
`git status`	what changed?
`git diff`	how exactly?
`git add <file>`	put a change into the next snapshot
`git commit -m "why"`	take the snapshot
`git log -oneline`	the history
`git restore <file>`	throw away uncommitted edits
`git switch -c <name>`	start a branch
`git push` / `git pull`	sync with GitHub

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

Git in your fingers

15 MINUTES, IN PAIRS, NO AGENT, NO GITHUB

	Exercise	Time
1	First commits: `init`, `status`, `add`, `commit`	4 min
2	Choose what goes into a commit	4 min
3	Undo, before and after committing	4 min
4	Read the history	3 min

Git in your fingers

15 MINUTES, IN PAIRS, NO AGENT, NO GITHUB

	Exercise	Time
1	First commits: `init`, `status`, `add`, `commit`	4 min
2	Choose what goes into a commit	4 min
3	Undo, before and after committing	4 min
4	Read the history	3 min

- A throw-away folder: nothing you do here can break anything

Git in your fingers

15 MINUTES, IN PAIRS, NO AGENT, NO GITHUB

	Exercise	Time
1	First commits: `init`, `status`, `add`, `commit`	4 min
2	Choose what goes into a commit	4 min
3	Undo, before and after committing	4 min
4	Read the history	3 min

- A throw-away folder: nothing you do here can break anything
- Run `git status` after every step and read what it says

Exercise 1: first commits

```
$ mkdir git-practice
$ cd git-practice
$ git init
$ git status                  # "No commits yet"
      ... create rabi.jl and notes.md in VS Code (content below) ...
$ git status                  # two untracked files
$ git add rabi.jl
$ git status                  # one staged, one untracked
$ git commit -m "Add the Rabi formula"
$ git add notes.md
$ git commit -m "Add notes"
$ git log --oneline           # two commits, newest first
```

Exercise 1: first commits

```
$ mkdir git-practice
$ cd git-practice
$ git init
$ git status                  # "No commits yet"
      ... create rabi.jl and notes.md in VS Code (content below) ...
$ git status                  # two untracked files
$ git add rabi.jl
$ git status                  # one staged, one untracked
$ git commit -m "Add the Rabi formula"
$ git add notes.md
$ git commit -m "Add notes"
$ git log --oneline           # two commits, newest first
```

- `rabi.jl`: the Rabi formula for $P_e(t)$, as one line of Julia

Exercise 1: first commits

```
$ mkdir git-practice
$ cd git-practice
$ git init
$ git status                  # "No commits yet"
      ... create rabi.jl and notes.md in VS Code (content below) ...
$ git status                  # two untracked files
$ git add rabi.jl
$ git status                  # one staged, one untracked
$ git commit -m "Add the Rabi formula"
$ git add notes.md
$ git commit -m "Add notes"
$ git log --oneline           # two commits, newest first
```

- `rabi.jl`: the Rabi formula for $P_e(t)$, as one line of Julia
- `notes.md`: anything, e.g. "$\hbar = 1$, rates in rad/µs"

Choosing what goes into a commit

THE FORMS OF `GIT ADD`

Command	Stages
`git add <file>`	One file
`git add .`	Everything new, changed or deleted in this folder and below
`git add -u`	Changes to files git already tracks; new files stay out
`git add -p`	Piece by piece: git shows each change and asks `y` (yes) or `n` (no)

Choosing what goes into a commit

THE FORMS OF `GIT ADD`

Command	Stages
`git add <file>`	One file
`git add .`	Everything new, changed or deleted in this folder and below
`git add -u`	Changes to files git already tracks; new files stay out
`git add -p`	Piece by piece: git shows each change and asks `y` (yes) or `n` (no)

Choosing what goes into a commit

THE FORMS OF `GIT ADD`

Command	Stages
`git add <file>`	One file
`git add .`	Everything new, changed or deleted in this folder and below
`git add -u`	Changes to files git already tracks; new files stay out
`git add -p`	Piece by piece: git shows each change and asks `y` (yes) or `n` (no)

Choosing what goes into a commit

THE FORMS OF `GIT ADD`

Command	Stages
`git add <file>`	One file
`git add .`	Everything new, changed or deleted in this folder and below
`git add -u`	Changes to files git already tracks; new files stay out
`git add -p`	Piece by piece: git shows each change and asks `y` (yes) or `n` (no)

Choosing what goes into a commit

THE FORMS OF `GIT ADD`

Command	Stages
`git add <file>`	One file
`git add .`	Everything new, changed or deleted in this folder and below
`git add -u`	Changes to files git already tracks; new files stay out
`git add -p`	Piece by piece: git shows each change and asks `y` (yes) or `n` (no)

`git commit -am "..."`: `git add -u`, then commit, in one step. Convenient, but you skip the look at what you stage

Exercise 2: one change, one commit

```
      ... edit BOTH files: add a line to notes.md, a comment to rabi.jl ...
$ git status                  # two modified files
$ git diff                    # what changed, line by line
$ git add -p                  # asks per file: n for notes.md, y for rabi.jl
$ git diff --staged           # exactly what will be committed
$ git commit -m "Document the units in rabi.jl"
$ git status                  # notes.md is still modified
$ git add -u
$ git commit -m "Extend the notes"
```

Exercise 2: one change, one commit

```
      ... edit BOTH files: add a line to notes.md, a comment to rabi.jl ...
$ git status                  # two modified files
$ git diff                    # what changed, line by line
$ git add -p                  # asks per file: n for notes.md, y for rabi.jl
$ git diff --staged           # exactly what will be committed
$ git commit -m "Document the units in rabi.jl"
$ git status                  # notes.md is still modified
$ git add -u
$ git commit -m "Extend the notes"
```

- Two unrelated changes, two commits: each one reviewable and revertible

Exercise 2: one change, one commit

```
      ... edit BOTH files: add a line to notes.md, a comment to rabi.jl ...
$ git status                  # two modified files
$ git diff                    # what changed, line by line
$ git add -p                  # asks per file: n for notes.md, y for rabi.jl
$ git diff --staged           # exactly what will be committed
$ git commit -m "Document the units in rabi.jl"
$ git status                  # notes.md is still modified
$ git add -u
$ git commit -m "Extend the notes"
```

- Two unrelated changes, two commits: each one reviewable and revertible
- `git diff` before `add`, `git diff -staged` before `commit`

Exercise 3: undo

```
      ... delete the formula in rabi.jl, save ...
$ git diff                    # the deletion, in red
$ git restore rabi.jl         # back to the last commit

      ... change rabi.jl again, then ...
$ git add rabi.jl
$ git restore --staged rabi.jl     # out of the staging area, edit kept
$ git restore rabi.jl              # edit gone too

      ... break the formula (t/2 → t), commit it ...
$ git revert --no-edit HEAD   # a new commit that undoes the last one
$ git log --oneline           # both commits are in the history
```

Exercise 3: undo

```
      ... delete the formula in rabi.jl, save ...
$ git diff                    # the deletion, in red
$ git restore rabi.jl         # back to the last commit

      ... change rabi.jl again, then ...
$ git add rabi.jl
$ git restore --staged rabi.jl     # out of the staging area, edit kept
$ git restore rabi.jl              # edit gone too

      ... break the formula (t/2 → t), commit it ...
$ git revert --no-edit HEAD   # a new commit that undoes the last one
$ git log --oneline           # both commits are in the history
```

- `HEAD`: the latest commit; any hash from `git log` works too

Exercise 4: read the history

```
$ git log --oneline           # one line per commit
$ git log --stat              # which files each commit touched
$ git show <hash>             # one commit in full: message and diff
$ git diff <hash1> <hash2>    # what changed between two snapshots
$ git log -p rabi.jl          # the whole story of one file
```

Exercise 4: read the history

```
$ git log --oneline           # one line per commit
$ git log --stat              # which files each commit touched
$ git show <hash>             # one commit in full: message and diff
$ git diff <hash1> <hash2>    # what changed between two snapshots
$ git log -p rabi.jl          # the whole story of one file
```

- Long output opens in a viewer: arrows to scroll, `q` to quit

Exercise 4: read the history

```
$ git log --oneline           # one line per commit
$ git log --stat              # which files each commit touched
$ git show <hash>             # one commit in full: message and diff
$ git diff <hash1> <hash2>    # what changed between two snapshots
$ git log -p rabi.jl          # the whole story of one file
```

- Long output opens in a viewer: arrows to scroll, `q` to quit
- The first 7 characters of a hash are enough

Exercise 4: read the history

```
$ git log --oneline           # one line per commit
$ git log --stat              # which files each commit touched
$ git show <hash>             # one commit in full: message and diff
$ git diff <hash1> <hash2>    # what changed between two snapshots
$ git log -p rabi.jl          # the whole story of one file
```

- Long output opens in a viewer: arrows to scroll, `q` to quit
- The first 7 characters of a hash are enough
- Same information in VS Code: Source Control panel, *Graph* view

Checkpoint

- See what changed: `git status`, `git diff`

Checkpoint

- See what changed: `git status`, `git diff`
- Choose what to commit: `git add`, `git diff --staged`

Checkpoint

- See what changed: `git status`, `git diff`
- Choose what to commit: `git add`, `git diff --staged`
- Undo before and after committing: `git restore`, `git revert`

Checkpoint

- See what changed: `git status`, `git diff`
- Choose what to commit: `git add`, `git diff --staged`
- Undo before and after committing: `git restore`, `git revert`
- Read the past: `git log`, `git show`

Checkpoint

- See what changed: `git status`, `git diff`
- Choose what to commit: `git add`, `git diff --staged`
- Undo before and after committing: `git restore`, `git revert`
- Read the past: `git log`, `git show`

That is all you need to supervise an agent

Break

15 MINUTES

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

Which code made Figure 3?

- `fit_final_v2_REALLY.m`, `analysis_new.jl`, `old/`
- Parameters edited by hand before each run

Which code made Figure 3?

- `fit_final_v2_REALLY.m`, `analysis_new.jl`, `old/`
- Parameters edited by hand before each run
- “It worked on my laptop last spring”

Which code made Figure 3?

- `fit_final_v2_REALLY.m`, `analysis_new.jl`, `old/`
- Parameters edited by hand before each run
- “It worked on my laptop last spring”
- An agent multiplies the files, and the confusion

Which code made Figure 3?

- `fit_final_v2_REALLY.m`, `analysis_new.jl`, `old/`
- Parameters edited by hand before each run
- “It worked on my laptop last spring”
- An agent multiplies the files, and the confusion

Goal: for every figure, you can say which code, data and parameters made it

One layout for every project

DRWATSON

JULIA> USING DRWATSON; INITIALIZE_PROJECT("DRIVENSPIN")

```
DrivenSpin/
|-- src/            # the physics: functions, no hard-coded parameters
|-- scripts/        # short runs that call src/ and save results
|-- test/           # runtests.jl: the physics checks
|-- data/
|   |-- exp_raw/    # measurements: read-only, never modified
|   |-- exp_pro/    # processed data
|   `-- sims/       # simulation results (generated)
|-- plots/          # figures (generated)
|-- Project.toml    # dependencies
|-- Manifest.toml   # exact versions
`-- AGENTS.md       # instructions for the agent
```

G. Datseris et al. *DrWatson: the perfect sidekick for your scientific inquiries*, J. Open Source Softw. **5**, 2673 (2020)

One layout for every project

DRWATSON

```
JULIA> USING DRWATSON; INITIALIZE_PROJECT("DRIVENSPIN")
```

```
DrivenSpin/
|-- src/            # the physics: functions, no hard-coded parameters
|-- scripts/        # short runs that call src/ and save results
|-- test/           # runtests.jl: the physics checks
|-- data/
|   |-- exp_raw/    # measurements: read-only, never modified
|   |-- exp_pro/    # processed data
|   `-- sims/       # simulation results (generated)
|-- plots/          # figures (generated)
|-- Project.toml    # dependencies
|-- Manifest.toml   # exact versions
`-- AGENTS.md       # instructions for the agent
```

- Same layout everywhere: you, your colleagues and the agent know where things go

G. Datseris et al. *DrWatson: the perfect sidekick for your scientific inquiries*, J. Open Source Softw. **5**, 2673 (2020)

One layout for every project

DRWATSON

JULIA> USING DRWATSON; INITIALIZE_PROJECT("DRIVENSPIN")

```
DrivenSpin/
|-- src/            # the physics: functions, no hard-coded parameters
|-- scripts/        # short runs that call src/ and save results
|-- test/           # runtests.jl: the physics checks
|-- data/
|   |-- exp_raw/    # measurements: read-only, never modified
|   |-- exp_pro/    # processed data
|   `-- sims/       # simulation results (generated)
|-- plots/          # figures (generated)
|-- Project.toml    # dependencies
|-- Manifest.toml   # exact versions
`-- AGENTS.md       # instructions for the agent
```

- Same layout everywhere: you, your colleagues and the agent know where things go
- Generated files can always be regenerated, so they stay out of git

G. Datseris et al. *DrWatson: the perfect sidekick for your scientific inquiries*, J. Open Source Softw. **5**, 2673 (2020)

The code is one part of the project

```
SpinProject/                  # everything about one research project
|-- DrivenSpin/               # the code: its own git repository
|   |-- .git/
|   `-- src/ scripts/ test/ data/ ...
|-- paper/                    # the manuscript: its own repository (e.g. Overleaf)
|-- notes/                    # derivations, meeting notes
`-- talks/
```

The code is one part of the project

```
SpinProject/                  # everything about one research project
|-- DrivenSpin/               # the code: its own git repository
|   |-- .git/
|   `-- src/ scripts/ test/ data/ ...
|-- paper/                    # the manuscript: its own repository (e.g. Overleaf)
|-- notes/                    # derivations, meeting notes
`-- talks/
```

- The code gets its own repository: shared, tested and published on its own

The code is one part of the project

```
SpinProject/                  # everything about one research project
|-- DrivenSpin/               # the code: its own git repository
|   |-- .git/
|   `-- src/ scripts/ test/ data/ ...
|-- paper/                    # the manuscript: its own repository (e.g. Overleaf)
|-- notes/                    # derivations, meeting notes
`-- talks/
```

- The code gets its own repository: shared, tested and published on its own
- Paper and talks have other collaborators and another history

The code is one part of the project

```
SpinProject/                  # everything about one research project
|-- DrivenSpin/               # the code: its own git repository
|   |-- .git/
|   `-- src/ scripts/ test/ data/ ...
|-- paper/                    # the manuscript: its own repository (e.g. Overleaf)
|-- notes/                    # derivations, meeting notes
`-- talks/
```

- The code gets its own repository: shared, tested and published on its own
- Paper and talks have other collaborators and another history
- Today, `DrivenSpin/` always means the code repository

`src` computes, scripts orchestrate

SCRIPTS/RABI.JL

```
using DrWatson
@quickactivate "DrivenSpin"          # this project's environment, from any folder
using DrivenSpin                     # the module in src/

for c in dict_list(Dict(:Δ => [0.0, 2π], :Ω => 2π))   # rad/μs; one run per Δ
    produce_or_load(c, datadir("sims", "rabi")) do c
        H = hamiltonian(c[:Δ], c[:Ω])
        t = range(0, 3, length = 301)                 # μs
        Dict("t" => t, "Pe" => excited_population.(Ref(H), t))
    end
end
```

`src` computes, scripts orchestrate

SCRIPTS/RABI.JL

```
using DrWatson
@quickactivate "DrivenSpin"          # this project's environment, from any folder
using DrivenSpin                     # the module in src/

for c in dict_list(Dict(:Δ => [0.0, 2π], :Ω => 2π))   # rad/μs; one run per Δ
    produce_or_load(c, datadir("sims", "rabi")) do c
        H = hamiltonian(c[:Δ], c[:Ω])
        t = range(0, 3, length = 301)                 # μs
        Dict("t" => t, "Pe" => excited_population.(Ref(H), t))
    end
end
```

- Scripts contain calls and parameters, no physics

`src` computes, scripts orchestrate

SCRIPTS/RABI.JL

```
using DrWatson
@quickactivate "DrivenSpin"          # this project's environment, from any folder
using DrivenSpin                     # the module in src/

for c in dict_list(Dict(:Δ => [0.0, 2π], :Ω => 2π))   # rad/μs; one run per Δ
    produce_or_load(c, datadir("sims", "rabi")) do c
        H = hamiltonian(c[:Δ], c[:Ω])
        t = range(0, 3, length = 301)                 # μs
        Dict("t" => t, "Pe" => excited_population.(Ref(H), t))
    end
end
```

- Scripts contain calls and parameters, no physics
- The same function serves the script, the test and the notebook

`src` computes, scripts orchestrate

SCRIPTS/RABI.JL

```
using DrWatson
@quickactivate "DrivenSpin"          # this project's environment, from any folder
using DrivenSpin                     # the module in src/

for c in dict_list(Dict(:Δ => [0.0, 2π], :Ω => 2π))   # rad/μs; one run per Δ
    produce_or_load(c, datadir("sims", "rabi")) do c
        H = hamiltonian(c[:Δ], c[:Ω])
        t = range(0, 3, length = 301)                 # μs
        Dict("t" => t, "Pe" => excited_population.(Ref(H), t))
    end
end
```

- Scripts contain calls and parameters, no physics
- The same function serves the script, the test and the notebook
- `produce_or_load`: compute and save, or load if the file exists

Every result knows its commit

```
$ ls data/sims/rabi
Δ=0.0_Ω=6.28.jld2    Δ=6.28_Ω=6.28.jld2
julia> data = wload(datadir("sims", "rabi", "Δ=0.0_Ω=6.28.jld2"));

julia> permutedims(sort(collect(keys(data))))
1×3 Matrix{String}:
 "Pe"  "gitcommit"  "t"

julia> data["gitcommit"]
"3f2a91c4e8...-dirty"
```

Every result knows its commit

```
$ ls data/sims/rabi
Δ=0.0_Ω=6.28.jld2    Δ=6.28_Ω=6.28.jld2
julia> data = wload(datadir("sims", "rabi", "Δ=0.0_Ω=6.28.jld2"));

julia> permutedims(sort(collect(keys(data))))
1×3 Matrix{String}:
 "Pe"  "gitcommit"  "t"

julia> data["gitcommit"]
"3f2a91c4e8...-dirty"
```

- File name = parameters; run the script again and it loads instead of computing

Every result knows its commit

```
$ ls data/sims/rabi
Δ=0.0_Ω=6.28.jld2    Δ=6.28_Ω=6.28.jld2
julia> data = wload(datadir("sims", "rabi", "Δ=0.0_Ω=6.28.jld2"));

julia> permutedims(sort(collect(keys(data))))
1×3 Matrix{String}:
 "Pe"  "gitcommit"  "t"

julia> data["gitcommit"]
"3f2a91c4e8...-dirty"
```

- File name = parameters; run the script again and it loads instead of computing
- `gitcommit`, added automatically: which version of the code made the file

Every result knows its commit

```
$ ls data/sims/rabi
Δ=0.0_Ω=6.28.jld2    Δ=6.28_Ω=6.28.jld2
julia> data = wload(datadir("sims", "rabi", "Δ=0.0_Ω=6.28.jld2"));

julia> permutedims(sort(collect(keys(data))))
1×3 Matrix{String}:
 "Pe"  "gitcommit"  "t"

julia> data["gitcommit"]
"3f2a91c4e8...-dirty"
```

- File name = parameters; run the script again and it loads instead of computing
- `gitcommit`, added automatically: which version of the code made the file
- `-dirty`: the code had uncommitted changes, so that version is lost

Every result knows its commit

```
$ ls data/sims/rabi
Δ=0.0_Ω=6.28.jld2    Δ=6.28_Ω=6.28.jld2
julia> data = wload(datadir("sims", "rabi", "Δ=0.0_Ω=6.28.jld2"));

julia> permutedims(sort(collect(keys(data))))
1×3 Matrix{String}:
 "Pe"  "gitcommit"  "t"

julia> data["gitcommit"]
"3f2a91c4e8...-dirty"
```

- File name = parameters; run the script again and it loads instead of computing
- `gitcommit`, added automatically: which version of the code made the file
- `-dirty`: the code had uncommitted changes, so that version is lost

Commit first, then run

The environment is part of the result

`Project.toml`

- What you depend on
- Written by `Pkg.add`

`Manifest.toml`

- Exact version of every package
- Never edited by hand

The environment is part of the result

`Project.toml`

- What you depend on
- Written by `Pkg.add`

`Manifest.toml`

- Exact version of every package
- Never edited by hand

```
$ git clone git@github.com:you/DrivenSpin.git
$ cd DrivenSpin
$ julia --project -e 'using Pkg; Pkg.instantiate()'    # same versions as yours
```

The environment is part of the result

`Project.toml`

- What you depend on
- Written by `Pkg.add`

`Manifest.toml`

- Exact version of every package
- Never edited by hand

```
$ git clone git@github.com:you/DrivenSpin.git
$ cd DrivenSpin
$ julia --project -e 'using Pkg; Pkg.instantiate()'    # same versions as yours
```

- Commit both: a colleague, a referee or CI gets your exact setup

The environment is part of the result

`Project.toml`

- What you depend on
- Written by `Pkg.add`

`Manifest.toml`

- Exact version of every package
- Never edited by hand

```
$ git clone git@github.com:you/DrivenSpin.git
$ cd DrivenSpin
$ julia --project -e 'using Pkg; Pkg.instantiate()'    # same versions as yours
```

- Commit both: a colleague, a referee or CI gets your exact setup
- Agents sometimes “fix” an error with `Pkg.update()`: package versions change, and your numbers may change with them

The environment is part of the result

`Project.toml`

- What you depend on
- Written by `Pkg.add`

`Manifest.toml`

- Exact version of every package
- Never edited by hand

```
$ git clone git@github.com:you/DrivenSpin.git
$ cd DrivenSpin
$ julia --project -e 'using Pkg; Pkg.instantiate()'    # same versions as yours
```

- Commit both: a colleague, a referee or CI gets your exact setup
- Agents sometimes “fix” an error with `Pkg.update()`: package versions change, and your numbers may change with them
- `Manifest.toml` in a diff you did not expect: ask why

A test is a claim you can run

TEST/RUNTESTS.JL

```
using DrivenSpin, Test

@testset "Rabi" begin
    Δ, Ω = 0.3, 2π                                  # rad/μs
    ΩR = sqrt(Ω^2 + Δ^2)
    H = hamiltonian(Δ, Ω)
    for t in range(0, 3, length = 31)               # μs
        @test excited_population(H, t) ≈ Ω^2 / ΩR^2 * sin(ΩR * t / 2)^2 atol = 1e-10
    end
    @test excited_population(hamiltonian(0.0, Ω), π / Ω) ≈ 1    # π-pulse
end
```

A test is a claim you can run

TEST/RUNTESTS.JL

```
using DrivenSpin, Test

@testset "Rabi" begin
    Δ, Ω = 0.3, 2π                                  # rad/μs
    ΩR = sqrt(Ω^2 + Δ^2)
    H = hamiltonian(Δ, Ω)
    for t in range(0, 3, length = 31)               # μs
        @test excited_population(H, t) ≈ Ω^2 / ΩR^2 * sin(ΩR * t / 2)^2 atol = 1e-10
    end
    @test excited_population(hamiltonian(0.0, Ω), π / Ω) ≈ 1    # π-pulse
end
```

- One `@testset` per physical claim; its name says which

A test is a claim you can run

TEST/RUNTESTS.JL

```
using DrivenSpin, Test

@testset "Rabi" begin
    Δ, Ω = 0.3, 2π                                  # rad/μs
    ΩR = sqrt(Ω^2 + Δ^2)
    H = hamiltonian(Δ, Ω)
    for t in range(0, 3, length = 31)               # μs
        @test excited_population(H, t) ≈ Ω^2 / ΩR^2 * sin(ΩR * t / 2)^2 atol = 1e-10
    end
    @test excited_population(hamiltonian(0.0, Ω), π / Ω) ≈ 1    # π-pulse
end
```

- One `@testset` per physical claim; its name says which
- `a ≈ b` compares with a *relative* tolerance: near zero, give `atol`

Running the tests

```
$ julia --project -e 'using Pkg; Pkg.test()'
     Testing Running tests...
Test Summary: | Pass  Total  Time
Rabi          |   32     32  0.4s
     Testing DrivenSpin tests passed
```

Running the tests

```
$ julia --project -e 'using Pkg; Pkg.test()'
     Testing Running tests...
Test Summary: | Pass  Total  Time
Rabi          |   32     32  0.4s
     Testing DrivenSpin tests passed
```

- Run them before every commit; the agent can run them too

Running the tests

```
$ julia --project -e 'using Pkg; Pkg.test()'
     Testing Running tests...
Test Summary: | Pass  Total  Time
Rabi          |   32     32  0.4s
     Testing DrivenSpin tests passed
```

- Run them before every commit; the agent can run them too
- Keep them fast: under a minute, or nobody runs them

Running the tests

```
$ julia --project -e 'using Pkg; Pkg.test()'
     Testing Running tests...
Test Summary: | Pass  Total  Time
Rabi          |   32     32  0.4s
     Testing DrivenSpin tests passed
```

- Run them before every commit; the agent can run them too
- Keep them fast: under a minute, or nobody runs them
- A failing test is information, not an obstacle

What makes a good test

- **Physics, not implementation**: a formula, a limit, a conservation law

What makes a good test

- **Physics, not implementation**: a formula, a limit, a conservation law
- **Deterministic**: fix the random seed

What makes a good test

- **Physics, not implementation**: a formula, a limit, a conservation law
- **Deterministic**: fix the random seed
- **Fast**: small systems, short times

What makes a good test

- **Physics, not implementation**: a formula, a limit, a conservation law
- **Deterministic**: fix the random seed
- **Fast**: small systems, short times
- **It fails when the physics is wrong**: break the code on purpose and check

What makes a good test

- **Physics, not implementation**: a formula, a limit, a conservation law
- **Deterministic**: fix the random seed
- **Fast**: small systems, short times
- **It fails when the physics is wrong**: break the code on purpose and check

A test that cannot fail checks nothing

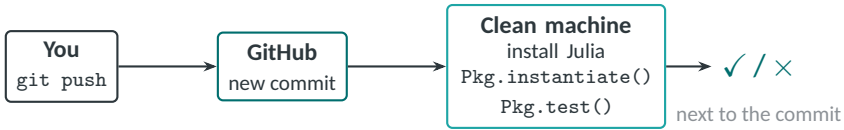
Continuous integration

THE TESTS RUN ON EVERY PUSH



Continuous integration

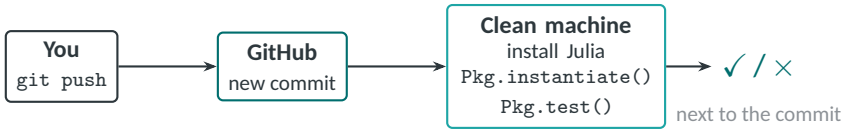
THE TESTS RUN ON EVERY PUSH



- A clean machine: catches “works on my laptop” and missing dependencies

Continuous integration

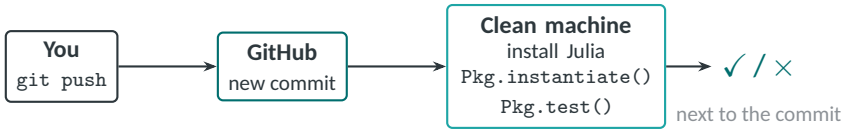
THE TESTS RUN ON EVERY PUSH



- A clean machine: catches “works on my laptop” and missing dependencies
- Nobody can forget to run the tests, including the agent

Continuous integration

THE TESTS RUN ON EVERY PUSH



- A clean machine: catches “works on my laptop” and missing dependencies
- Nobody can forget to run the tests, including the agent
- A referee who never sleeps and reads every commit

The whole configuration

.GITHUB/WORKFLOWS/CI.YML

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: julia-actions/setup-julia@v2
      - uses: julia-actions/cache@v2
      - uses: julia-actions/julia-buildpkg@v1
      - uses: julia-actions/julia-runtest@v1
```

The whole configuration

.GITHUB/WORKFLOWS/CI.YML

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: julia-actions/setup-julia@v2
      - uses: julia-actions/cache@v2
      - uses: julia-actions/julia-buildpkg@v1
      - uses: julia-actions/julia-runtest@v1
```

- Copy it once; it works for any Julia project

The whole configuration

.GITHUB/WORKFLOWS/CI.YML

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: julia-actions/setup-julia@v2
      - uses: julia-actions/cache@v2
      - uses: julia-actions/julia-buildpkg@v1
      - uses: julia-actions/julia-runtest@v1
```

- Copy it once; it works for any Julia project
- Results under the *Actions* tab of the repository on GitHub

Your tools, same principles

MATLAB, INSTRUMENTS, FPGA

Principle	In your lab
Version control	Analysis scripts, instrument settings, HDL sources
Tests	`matlab.unittest`; for FPGA designs, simulation testbenches
Raw data is read-only	The agent never writes where measurements land
Traceability	Save the commit hash and settings with every measurement run

Your tools, same principles

MATLAB, INSTRUMENTS, FPGA

Principle	In your lab
Version control	Analysis scripts, instrument settings, HDL sources
Tests	`matlab.unittest`; for FPGA designs, simulation testbenches
Raw data is read-only	The agent never writes where measurements land
Traceability	Save the commit hash and settings with every measurement run

Your tools, same principles

MATLAB, INSTRUMENTS, FPGA

Principle	In your lab
Version control	Analysis scripts, instrument settings, HDL sources
Tests	`matlab.unittest`; for FPGA designs, simulation testbenches
Raw data is read-only	The agent never writes where measurements land
Traceability	Save the commit hash and settings with every measurement run

Your tools, same principles

MATLAB, INSTRUMENTS, FPGA

Principle	In your lab
Version control	Analysis scripts, instrument settings, HDL sources
Tests	`matlab.unittest`; for FPGA designs, simulation testbenches
Raw data is read-only	The agent never writes where measurements land
Traceability	Save the commit hash and settings with every measurement run

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

Physics gives you the checks

EXACT STATEMENTS

Check	Driven spin
Analytic solution	$P_e(t) = (\Omega^2/\Omega_R^2)\sin^2(\Omega_R t/2)$
Limiting case	$\Omega = 0$: nothing moves; $\Delta = 0$, $t = \pi/\Omega$: full inversion
Structure	$\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues of $\rho \geq 0$
Symmetry	P_e is even in Δ

P_e: excited-state population, starting in $|g\rangle$; $\Omega_R = \sqrt{\Omega^2 + \Delta^2}$; ρ: density matrix

Physics gives you the checks

EXACT STATEMENTS

Check	Driven spin
Analytic solution	$P_e(t) = (\Omega^2/\Omega_R^2)\sin^2(\Omega_R t/2)$
Limiting case	$\Omega = 0$: nothing moves; $\Delta = 0$, $t = \pi/\Omega$: full inversion
Structure	$\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues of $\rho \geq 0$
Symmetry	P_e is even in Δ

P_e: excited-state population, starting in $|g\rangle$; $\Omega_R = \sqrt{\Omega^2 + \Delta^2}$; ρ: density matrix

Physics gives you the checks

EXACT STATEMENTS

Check	Driven spin
Analytic solution	$P_e(t) = (\Omega^2/\Omega_R^2)\sin^2(\Omega_R t/2)$
Limiting case	$\Omega = 0$: nothing moves; $\Delta = 0$, $t = \pi/\Omega$: full inversion
Structure	$\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues of $\rho \geq 0$
Symmetry	P_e is even in Δ

P_e: excited-state population, starting in $|g\rangle$; $\Omega_R = \sqrt{\Omega^2 + \Delta^2}$; ρ: density matrix

Physics gives you the checks

EXACT STATEMENTS

Check	Driven spin
Analytic solution	$P_e(t) = (\Omega^2/\Omega_R^2)\sin^2(\Omega_R t/2)$
Limiting case	$\Omega = 0$: nothing moves; $\Delta = 0$, $t = \pi/\Omega$: full inversion
Structure	$\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues of $\rho \geq 0$
Symmetry	P_e is even in Δ

P_e: excited-state population, starting in $|g\rangle$; $\Omega_R = \sqrt{\Omega^2 + \Delta^2}$; ρ: density matrix

Physics gives you the checks

CONSISTENCY

Check	Driven spin
Convergence	The result does not change when you halve the time step
Independent route	Your solver against QuantumOptics.jl
Bounds and units	$0 \leq P_e \leq 1$; $T_2 \leq 2T_1$; rates in rad/µs
Injection–recovery	Simulate data with a known T_2, fit it: you get T_2 back

T_1: energy relaxation time; T_2: coherence time

Physics gives you the checks

CONSISTENCY

Check	Driven spin
Convergence	The result does not change when you halve the time step
Independent route	Your solver against QuantumOptics.jl
Bounds and units	$0 \leq P_e \leq 1$; $T_2 \leq 2T_1$; rates in rad/µs
Injection–recovery	Simulate data with a known T_2, fit it: you get T_2 back

T_1: energy relaxation time; T_2: coherence time

Physics gives you the checks

CONSISTENCY

Check	Driven spin
Convergence	The result does not change when you halve the time step
Independent route	Your solver against QuantumOptics.jl
Bounds and units	$0 \leq P_e \leq 1$; $T_2 \leq 2T_1$; rates in rad/µs
Injection–recovery	Simulate data with a known T_2, fit it: you get T_2 back

T_1: energy relaxation time; T_2: coherence time

Physics gives you the checks

CONSISTENCY

Check	Driven spin
Convergence	The result does not change when you halve the time step
Independent route	Your solver against QuantumOptics.jl
Bounds and units	$0 \leq P_e \leq 1$; $T_2 \leq 2T_1$; rates in rad/µs
Injection–recovery	Simulate data with a known T_2, fit it: you get T_2 back

T_1: energy relaxation time; T_2: coherence time

Convergence

IS THE NUMBER CONVERGED, AND IS THE METHOD WHAT IT CLAIMS?

$$\varepsilon(\delta t) \simeq C\,\delta t^{p} \quad \Longrightarrow \quad \frac{\varepsilon(\delta t)}{\varepsilon(\delta t/2)} \simeq 2^{p}$$

```
ρT(dt) = last(lindblad(H, ρ0, [T]; T1, T2, dt))           # fixed-step RK4
err(dt) = abs(excited_population(ρT(dt)) - exp(-T / T1))  # free decay from |e⟩
@test log2(err(0.02) / err(0.01)) ≈ 4 atol = 0.2          # RK4: p = 4
```

- ε: error at fixed time T; δt: step; p: order of the method

Convergence

IS THE NUMBER CONVERGED, AND IS THE METHOD WHAT IT CLAIMS?

$$\varepsilon(\delta t) \simeq C\,\delta t^{p} \quad \Longrightarrow \quad \frac{\varepsilon(\delta t)}{\varepsilon(\delta t/2)} \simeq 2^{p}$$

```
ρT(dt) = last(lindblad(H, ρ0, [T]; T1, T2, dt))           # fixed-step RK4
err(dt) = abs(excited_population(ρT(dt)) - exp(-T / T1))  # free decay from |e⟩
@test log2(err(0.02) / err(0.01)) ≈ 4 atol = 0.2          # RK4: p = 4
```

- ε: error at fixed time T; δt: step; p: order of the method
- Catches a wrong integrator, a missing term, an unconverged result

Convergence

IS THE NUMBER CONVERGED, AND IS THE METHOD WHAT IT CLAIMS?

$$\varepsilon(\delta t) \simeq C\,\delta t^{p} \quad \Longrightarrow \quad \frac{\varepsilon(\delta t)}{\varepsilon(\delta t/2)} \simeq 2^{p}$$

```
ρT(dt) = last(lindblad(H, ρ0, [T]; T1, T2, dt))           # fixed-step RK4
err(dt) = abs(excited_population(ρT(dt)) - exp(-T / T1))  # free decay from |e⟩
@test log2(err(0.02) / err(0.01)) ≈ 4 atol = 0.2          # RK4: p = 4
```

- ε: error at fixed time T; δt: step; p: order of the method
- Catches a wrong integrator, a missing term, an unconverged result
- Same idea for any cutoff: grid size, Fock space, number of samples

Two independent routes

```
using QuantumOptics
b = SpinBasis(1//2)                                    # |e⟩ = spin up
H = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
c_ops = [sqrt(1/T1)*sigmam(b), sqrt(1/(2Tϕ))*sigmaz(b)]
_, ρt = timeevolution.master(ts, dm(spindown(b)), H, c_ops)
@test [real(expect(projector(spinup(b)), ρ)) for ρ in ρt] ≈ my_Pe  rtol = 1e-6
```

Two independent routes

```
using QuantumOptics
b = SpinBasis(1//2)                                    # |e⟩ = spin up
H = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
c_ops = [sqrt(1/T1)*sigmam(b), sqrt(1/(2Tϕ))*sigmaz(b)]
_, ρt = timeevolution.master(ts, dm(spindown(b)), H, c_ops)
@test [real(expect(projector(spinup(b)), ρ)) for ρ in ρt] ≈ my_Pe  rtol = 1e-6
```

- Different method, different author: their errors are unlikely to coincide

Two independent routes

```
using QuantumOptics
b = SpinBasis(1//2)                                    # |e⟩ = spin up
H = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
c_ops = [sqrt(1/T1)*sigmam(b), sqrt(1/(2Tϕ))*sigmaz(b)]
_, ρt = timeevolution.master(ts, dm(spindown(b)), H, c_ops)
@test [real(expect(projector(spinup(b)), ρ)) for ρ in ρt] ≈ my_Pe  rtol = 1e-6
```

- Different method, different author: their errors are unlikely to coincide
- Disagreement does not tell you who is wrong; agreement is strong evidence

Two independent routes

```
using QuantumOptics
b = SpinBasis(1//2)                                    # |e⟩ = spin up
H = Δ/2*sigmaz(b) + Ω/2*sigmax(b)
c_ops = [sqrt(1/T1)*sigmam(b), sqrt(1/(2Tϕ))*sigmaz(b)]
_, ρt = timeevolution.master(ts, dm(spindown(b)), H, c_ops)
@test [real(expect(projector(spinup(b)), ρ)) for ρ in ρt] ≈ my_Pe  rtol = 1e-6
```

- Different method, different author: their errors are unlikely to coincide
- Disagreement does not tell you who is wrong; agreement is strong evidence
- Also for derivations: a numerical check of every analytic result

Injection–recovery

THE EXPERIMENTALIST'S TEST



$$\left|\hat{\theta} - \theta^*\right| < 3\sigma$$

- θ^*: true parameters; $\hat{\theta}$: fitted values; σ: reported errors

Injection–recovery

THE EXPERIMENTALIST'S TEST



$$\left|\hat{\theta} - \theta^*\right| < 3\sigma$$

- θ^*: true parameters; $\hat{\theta}$: fitted values; σ: reported errors
- Tests the whole pipeline: model, noise, fit and error bars

Injection–recovery

THE EXPERIMENTALIST'S TEST

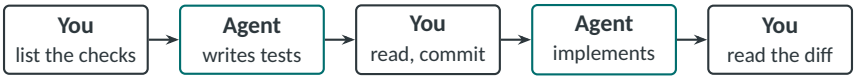


$$\left|\hat{\theta} - \theta^*\right| < 3\sigma$$

- θ^*: true parameters; $\hat{\theta}$: fitted values; σ: reported errors
- Tests the whole pipeline: model, noise, fit and error bars
- Over many seeds, $(\hat{\theta} - \theta^*)/\sigma$ must look like a unit Gaussian: are the error bars honest?

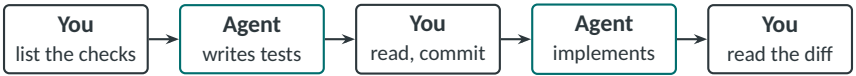
Tests first

THE TESTS ARE THE SPECIFICATION YOU SIGN



Tests first

THE TESTS ARE THE SPECIFICATION YOU SIGN



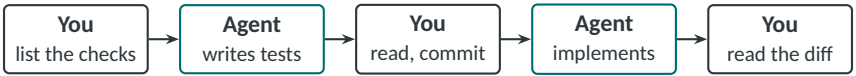
PROMPT

```
Implement lindblad() in src/DrivenSpin.jl until all tests pass.
Do not modify anything in test/. If a test seems wrong, stop and tell me why.
```

- Committed tests: a fixed target the agent cannot move unnoticed

Tests first

THE TESTS ARE THE SPECIFICATION YOU SIGN



PROMPT

```
Implement lindblad() in src/DrivenSpin.jl until all tests pass.
Do not modify anything in test/. If a test seems wrong, stop and tell me why.
```

- Committed tests: a fixed target the agent cannot move unnoticed
- Writing them first forces you to say what “correct” means

Who checks the tests?

- An agent writing the code and its tests grades its own homework

Who checks the tests?

- An agent writing the code and its tests grades its own homework
- Tests copied from the implementation check nothing: test against *physics*, not against the code

Who checks the tests?

- An agent writing the code and its tests grades its own homework
- Tests copied from the implementation check nothing: test against *physics*, not against the code
- Break the code on purpose: every important test must fail

Who checks the tests?

- An agent writing the code and its tests grades its own homework
- Tests copied from the implementation check nothing: test against *physics*, not against the code
- Break the code on purpose: every important test must fail
- You read every line of `test/`; it is short

Who checks the tests?

- An agent writing the code and its tests grades its own homework
- Tests copied from the implementation check nothing: test against *physics*, not against the code
- Break the code on purpose: every important test must fail
- You read every line of `test/`; it is short

The tests are the part you cannot delegate

Every claim gets a status

FROM MY OWN PROJECTS

Status	Meaning
verified	Reproduced by a second, independent route
fixed	Wrong as stated; corrected
weakened	True only under a narrower condition, now stated
retracted	Does not survive
open	Not decidable with the tools at hand

Every claim gets a status

FROM MY OWN PROJECTS

Status	Meaning
verified	Reproduced by a second, independent route
fixed	Wrong as stated; corrected
weakened	True only under a narrower condition, now stated
retracted	Does not survive
open	Not decidable with the tools at hand

Every claim gets a status

FROM MY OWN PROJECTS

Status	Meaning
verified	Reproduced by a second, independent route
fixed	Wrong as stated; corrected
weakened	True only under a narrower condition, now stated
retracted	Does not survive
open	Not decidable with the tools at hand

Every claim gets a status

FROM MY OWN PROJECTS

Status	Meaning
verified	Reproduced by a second, independent route
fixed	Wrong as stated; corrected
weakened	True only under a narrower condition, now stated
retracted	Does not survive
open	Not decidable with the tools at hand

Every claim gets a status

FROM MY OWN PROJECTS

Status	Meaning
verified	Reproduced by a second, independent route
fixed	Wrong as stated; corrected
weakened	True only under a narrower condition, now stated
retracted	Does not survive
open	Not decidable with the tools at hand

Every claim gets a status

FROM MY OWN PROJECTS

Status	Meaning
verified	Reproduced by a second, independent route
fixed	Wrong as stated; corrected
weakened	True only under a narrower condition, now stated
retracted	Does not survive
open	Not decidable with the tools at hand

Every result an agent produces gets one, in a file in the repository

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

The plan for the next hour

Stage	Physics		Time
1	Rabi oscillations	done: your starting point	—
2	Decoherence: T_1 and T_2		20 min
3	Ramsey fringes and spin echo		15 min
4	Synthetic experiment and fit		15 min
5	Stretch: cavity, CI, the planted bug		—

The plan for the next hour

Stage	Physics		Time
1	Rabi oscillations	done: your starting point	—
2	Decoherence: T_1 and T_2		20 min
3	Ramsey fringes and spin echo		15 min
4	Synthetic experiment and fit		15 min
5	Stretch: cavity, CI, the planted bug		—

- Pairs: the person new to git types the git commands

The plan for the next hour

Stage	Physics		Time
1	Rabi oscillations	done: your starting point	—
2	Decoherence: T_1 and T_2		20 min
3	Ramsey fringes and spin echo		15 min
4	Synthetic experiment and fit		15 min
5	Stretch: cavity, CI, the planted bug		—

- Pairs: the person new to git types the git commands
- Swap the keyboard after every stage

Getting started

```
$ git clone git@github.com:marcelojbp/TODO.git course     # course material
$ cp -r course/DrivenSpin DrivenSpin                       # your own copy
$ cd DrivenSpin
$ git init
$ git add .
$ git commit -m "Start from stage 1"
$ julia --project -e 'using Pkg; Pkg.instantiate(); Pkg.test()'
```

Getting started

```
$ git clone git@github.com:marcelojbp/TODO.git course     # course material
$ cp -r course/DrivenSpin DrivenSpin                       # your own copy
$ cd DrivenSpin
$ git init
$ git add .
$ git commit -m "Start from stage 1"
$ julia --project -e 'using Pkg; Pkg.instantiate(); Pkg.test()'
```

- Your own repository: your history, your commits

Getting started

```
$ git clone git@github.com:marcelojbp/TODO.git course     # course material
$ cp -r course/DrivenSpin DrivenSpin                       # your own copy
$ cd DrivenSpin
$ git init
$ git add .
$ git commit -m "Start from stage 1"
$ julia --project -e 'using Pkg; Pkg.instantiate(); Pkg.test()'
```

- Your own repository: your history, your commits
- The tests of stage 1 pass before you change anything

Getting started

```
$ git clone git@github.com:marcelojbp/TODO.git course     # course material
$ cp -r course/DrivenSpin DrivenSpin                       # your own copy
$ cd DrivenSpin
$ git init
$ git add .
$ git commit -m "Start from stage 1"
$ julia --project -e 'using Pkg; Pkg.instantiate(); Pkg.test()'
```

- Your own repository: your history, your commits
- The tests of stage 1 pass before you change anything
- Start your agent *inside* `DrivenSpin/`; it reads `AGENTS.md`

Each stage in four steps

PROMPT

```
1. Read AGENTS.md and src/. Write ONLY the tests for stage 2 in
   test/runtests.jl, from the checks on the slide. Do not implement anything.
```

- Tests first, from the physics on the slide

Each stage in four steps

```
2. $ git diff test/              # you read every line
   $ git commit -am "Stage 2: tests for T1 and T2 decay"
```

- Tests first, from the physics on the slide
- You read and commit the tests: now they are a fixed target

Each stage in four steps

PROMPT

```
3. Implement stage 2 in src/ until all tests pass.
   Do not modify anything in test/. If a test seems wrong, stop and tell me why.
```

- Tests first, from the physics on the slide
- You read and commit the tests: now they are a fixed target
- The agent works against the target, not around it

Each stage in four steps

```
4. $ git diff                    # you read the implementation
   $ julia --project -e 'using Pkg; Pkg.test()'
   $ git commit -am "Stage 2: Lindblad evolution with T1 and T2"
```

- Tests first, from the physics on the slide
- You read and commit the tests: now they are a fixed target
- The agent works against the target, not around it
- The diff is yours to approve

Stage 2: decoherence

20 MINUTES

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -i[H,\rho] + \Gamma_1 \mathcal{D}[\sigma_-]\rho + \frac{\Gamma_\phi}{2}\mathcal{D}[\sigma_z]\rho$$

$\mathcal{D}[L]\rho = L\rho L^\dagger - \frac{1}{2}\{L^\dagger L, \rho\}$; $\sigma_- = |g\rangle\langle e|$; $\Gamma_1 = 1/T_1, \Gamma_\phi = 1/T_\phi$; $1/T_2 = 1/(2T_1) + 1/T_\phi$

Tests

Stage 2: decoherence

20 MINUTES

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -i[H,\rho] + \Gamma_1 \mathcal{D}[\sigma_-]\rho + \frac{\Gamma_\phi}{2}\mathcal{D}[\sigma_z]\rho$$

$\mathcal{D}[L]\rho = L\rho L^\dagger - \frac{1}{2}\{L^\dagger L, \rho\}$; $\sigma_- = |g\rangle\langle e|$; $\Gamma_1 = 1/T_1, \Gamma_\phi = 1/T_\phi$; $1/T_2 = 1/(2T_1) + 1/T_\phi$

Tests

- At all times: $\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues ≥ 0

Stage 2: decoherence

20 MINUTES

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -i[H,\rho] + \Gamma_1 \mathcal{D}[\sigma_-]\rho + \frac{\Gamma_\phi}{2}\mathcal{D}[\sigma_z]\rho$$

$\mathcal{D}[L]\rho = L\rho L^\dagger - \frac{1}{2}\{L^\dagger L, \rho\}$; $\sigma_- = |g\rangle\langle e|$; $\Gamma_1 = 1/T_1, \Gamma_\phi = 1/T_\phi$; $1/T_2 = 1/(2T_1) + 1/T_\phi$

Tests

- At all times: $\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues ≥ 0
- $\Omega = 0$, start in $|e\rangle$: $P_e(t) = e^{-t/T_1}$

Stage 2: decoherence

20 MINUTES

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -i[H,\rho] + \Gamma_1 \mathcal{D}[\sigma_-]\rho + \frac{\Gamma_\phi}{2}\mathcal{D}[\sigma_z]\rho$$

$\mathcal{D}[L]\rho = L\rho L^\dagger - \frac{1}{2}\{L^\dagger L, \rho\}$; $\sigma_- = |g\rangle\langle e|$; $\Gamma_1 = 1/T_1, \Gamma_\phi = 1/T_\phi$; $1/T_2 = 1/(2T_1) + 1/T_\phi$

Tests

- At all times: $\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues ≥ 0
- $\Omega = 0$, start in $|e\rangle$: $P_e(t) = \mathrm{e}^{-t/T_1}$
- $\Omega = \Delta = 0$, start in $|+\rangle$: $|\rho_{eg}(t)| = \frac{1}{2}\mathrm{e}^{-t/T_2}$

$|+\rangle = (|e\rangle + |g\rangle)/\sqrt{2}$; $\rho_{eg} = \langle e|\,\rho\,|g\rangle$

Stage 2: decoherence

20 MINUTES

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -i[H,\rho] + \Gamma_1 \mathcal{D}[\sigma_-]\rho + \frac{\Gamma_\phi}{2}\mathcal{D}[\sigma_z]\rho$$

$\mathcal{D}[L]\rho = L\rho L^\dagger - \frac{1}{2}\{L^\dagger L, \rho\}$; $\sigma_- = |g\rangle\langle e|$; $\Gamma_1 = 1/T_1, \Gamma_\phi = 1/T_\phi$; $1/T_2 = 1/(2T_1) + 1/T_\phi$

Tests

- At all times: $\mathrm{tr}\rho = 1$, $\rho = \rho^\dagger$, eigenvalues ≥ 0
- $\Omega = 0$, start in $|e\rangle$: $P_e(t) = \mathrm{e}^{-t/T_1}$
- $\Omega = \Delta = 0$, start in $|+\rangle$: $|\rho_{eg}(t)| = \frac{1}{2}\mathrm{e}^{-t/T_2}$
- $T_2 > 2T_1$ is rejected with an error; $T_1, T_2 \to \infty$ gives stage 1

$|+\rangle = (|e\rangle + |g\rangle)/\sqrt{2}$; $\rho_{eg} = \langle e|\,\rho\,|g\rangle$

Stage 3: Ramsey and spin echo

15 MINUTES

$$P_e(\tau) = \tfrac{1}{2}\left[1 + e^{-\tau/T_2}\cos(\Delta\tau)\right]$$

Ramsey: $\pi/2$ pulse, free evolution for a time τ, $\pi/2$ pulse; ideal pulses $R(\theta) = e^{-i\theta\sigma_x/2}$; start in $|g\rangle$; $T_1 \to \infty$

Tests

Stage 3: Ramsey and spin echo

15 MINUTES

$$P_e(\tau) = \tfrac{1}{2}\left[1 + e^{-\tau/T_2}\cos(\Delta\tau)\right]$$

Ramsey: $\pi/2$ pulse, free evolution for a time τ, $\pi/2$ pulse; ideal pulses $R(\theta) = e^{-i\theta\sigma_x/2}$; start in $|g\rangle$; $T_1 \to \infty$

Tests

- Fringes oscillate at angular frequency Δ, under the envelope $e^{-\tau/T_2}$

Stage 3: Ramsey and spin echo

15 MINUTES

$$P_e(\tau) = \tfrac{1}{2}\left[1 + e^{-\tau/T_2}\cos(\Delta\tau)\right]$$

Ramsey: $\pi/2$ pulse, free evolution for a time τ, $\pi/2$ pulse; ideal pulses $R(\theta) = e^{-i\theta\sigma_x/2}$; start in $|g\rangle$; $T_1 \to \infty$

Tests

- Fringes oscillate at angular frequency Δ, under the envelope $e^{-\tau/T_2}$
- Echo, $\pi/2$–$\tau/2$–π–$\tau/2$–$\pi/2$: result independent of a static detuning δ

Stage 3: Ramsey and spin echo

15 MINUTES

$$P_e(\tau) = \tfrac{1}{2}\left[1 + e^{-\tau/T_2}\cos(\Delta\tau)\right]$$

Ramsey: $\pi/2$ pulse, free evolution for a time τ, $\pi/2$ pulse; ideal pulses $R(\theta) = e^{-i\theta\sigma_x/2}$; start in $|g\rangle$; $T_1 \to \infty$

Tests

- Fringes oscillate at angular frequency Δ, under the envelope $e^{-\tau/T_2}$
- Echo, $\pi/2$–$\tau/2$–π–$\tau/2$–$\pi/2$: result independent of a static detuning δ
- Average over random static δ: Ramsey decays, echo does not

Stage 4: a synthetic experiment

15 MINUTES

$$P(\tau) = B + A\,\mathrm{e}^{-\tau/T_2}\cos(\Delta\tau + \varphi)$$

Fit model for Ramsey data; A, B, φ: amplitude, offset, phase. Data: stage 3 with N measurements per point, binomial noise

Tests

Stage 4: a synthetic experiment

15 MINUTES

$$P(\tau) = B + A\,\mathrm{e}^{-\tau/T_2}\cos(\Delta\tau + \varphi)$$

Fit model for Ramsey data; A, B, φ: amplitude, offset, phase. Data: stage 3 with N measurements per point, binomial noise

Tests

- Injection–recovery: fitted T_2 and Δ within 3σ of the true values, fixed seed

Stage 4: a synthetic experiment

15 MINUTES

$$P(\tau) = B + A\,\mathrm{e}^{-\tau/T_2}\cos(\Delta\tau + \varphi)$$

Fit model for Ramsey data; A, B, φ: amplitude, offset, phase. Data: stage 3 with N measurements per point, binomial noise

Tests

- Injection–recovery: fitted T_2 and Δ within 3σ of the true values, fixed seed
- Four times more measurements: error bars about half as large

Stage 4: a synthetic experiment

15 MINUTES

$$P(\tau) = B + A\,\mathrm{e}^{-\tau/T_2}\cos(\Delta\tau + \varphi)$$

Fit model for Ramsey data; A, B, φ: amplitude, offset, phase. Data: stage 3 with N measurements per point, binomial noise

Tests

- Injection–recovery: fitted T_2 and Δ within 3σ of the true values, fixed seed
- Four times more measurements: error bars about half as large
- Script: save the fit with `produce_or_load`; the file carries the commit

Done early?

STRETCH GOALS

- **Push**: `gh repo create DrivenSpin -private -source=. -push`, then add the CI file and watch the *Actions* tab

Done early?

STRETCH GOALS

- **Push**: `gh repo create DrivenSpin -private -source=. -push`, then add the CI file and watch the *Actions* tab
- **Independent route**: check stage 2 against QuantumOptics.jl

Done early?

STRETCH GOALS

- **Push**: `gh repo create DrivenSpin -private -source=. -push`, then add the CI file and watch the *Actions* tab
- **Independent route**: check stage 2 against QuantumOptics.jl
- **Cavity**: Jaynes–Cummings, vacuum Rabi splitting $2g\sqrt{n+1}$, convergence in the photon cutoff

g: atom–cavity coupling; n: photon number

Done early?

STRETCH GOALS

- **Push**: `gh repo create DrivenSpin -private -source=. -push`, then add the CI file and watch the *Actions* tab
- **Independent route**: check stage 2 against QuantumOptics.jl
- **Cavity**: Jaynes–Cummings, vacuum Rabi splitting $2g\sqrt{n+1}$, convergence in the photon cutoff
- **Convergence**: test the order of your time stepper

g: atom–cavity coupling; n: photon number

When you are stuck

PROMPT

```
The test "free decay" fails. Run only that testset, read the failing
assertion, and explain the cause before changing any code.
```

When you are stuck

PROMPT

```
The test "free decay" fails. Run only that testset, read the failing
assertion, and explain the cause before changing any code.
```

- Ask for an explanation before a fix

When you are stuck

PROMPT

```
The test "free decay" fails. Run only that testset, read the failing
assertion, and explain the cause before changing any code.
```

- Ask for an explanation before a fix
- Agent went in circles? `git restore .` and a fresh session

When you are stuck

PROMPT

```
The test "free decay" fails. Run only that testset, read the failing
assertion, and explain the cause before changing any code.
```

- Ask for an explanation before a fix
- Agent went in circles? `git restore .` and a fresh session
- Still stuck: call me; that is what I am here for

Outline

- How a large language model works
- Chatbots and agents
- Context, effort and persistent instructions
- Git: the safety net
- Hands-on: git on your own machine
- Project structure, tests and CI
- Verification in physics
- Hands-on: the driven spin
- Failure modes and responsibility

The planted bug

FIVE MINUTES, ON YOUR OWN REPOSITORY

```
$ git apply ../course/DrivenSpin/exercises/planted_bug.patch
$ julia --project scripts/rabi.jl        # make the plot: does it look right?
$ julia --project -e 'using Pkg; Pkg.test()'
```

The planted bug

FIVE MINUTES, ON YOUR OWN REPOSITORY

```
$ git apply ../course/DrivenSpin/exercises/planted_bug.patch
$ julia --project scripts/rabi.jl        # make the plot: does it look right?
$ julia --project -e 'using Pkg; Pkg.test()'
```

- A one-character change, the kind an agent makes while “tidying up”

The planted bug

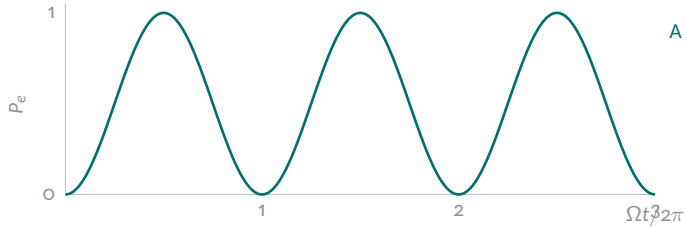
FIVE MINUTES, ON YOUR OWN REPOSITORY

```
$ git apply ../course/DrivenSpin/exercises/planted_bug.patch
$ julia --project scripts/rabi.jl        # make the plot: does it look right?
$ julia --project -e 'using Pkg; Pkg.test()'
```

- A one-character change, the kind an agent makes while “tidying up”
- Find it with `git diff`; undo it with `git restore .`

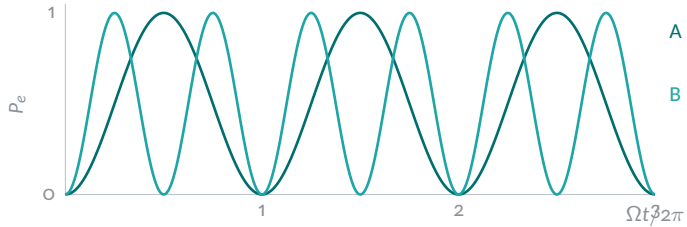
Plausible is not correct

$\Delta = 0$: WHICH CURVE IS RIGHT?



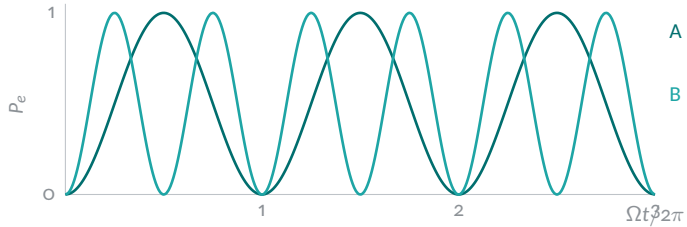
Plausible is not correct

$\Delta = 0$: WHICH CURVE IS RIGHT?



Plausible is not correct

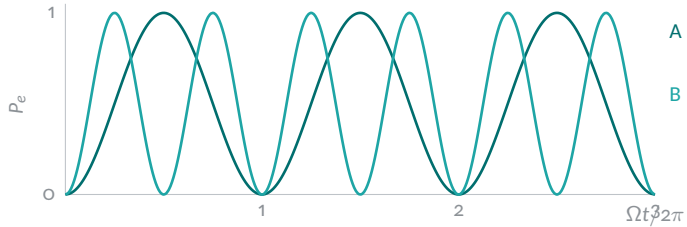
$\Delta = 0$: WHICH CURVE IS RIGHT?



- Both are smooth, bounded, periodic: both “look like Rabi oscillations”

Plausible is not correct

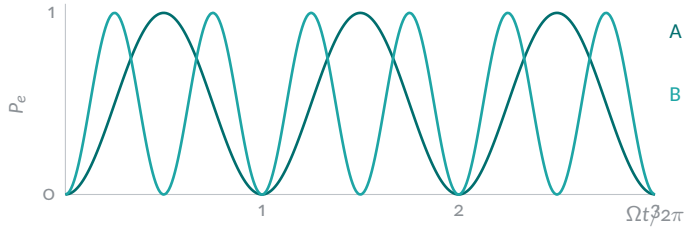
$\Delta = 0$: WHICH CURVE IS RIGHT?



- Both are smooth, bounded, periodic: both “look like Rabi oscillations”
- A is $\sin^2(\Omega t/2)$: correct. B is the bug, Ω instead of $\Omega/2$

Plausible is not correct

$\Delta = 0$: WHICH CURVE IS RIGHT?



- Both are smooth, bounded, periodic: both “look like Rabi oscillations”
- A is $\sin^2(\Omega t/2)$: correct. B is the bug, Ω instead of $\Omega/2$
- The eye checks shape; the test checks the number

The bug your tests cannot see

A SIGN IN THE HAMILTONIAN TERM

$$-i[H, \rho] \longrightarrow +i[H, \rho]$$

The bug your tests cannot see

A SIGN IN THE HAMILTONIAN TERM

$$-i[H, \rho] \longrightarrow +i[H, \rho]$$

- Every population is unchanged: Rabi, decay, Ramsey and echo tests all pass

The bug your tests cannot see

A SIGN IN THE HAMILTONIAN TERM

$$-i[H, \rho] \longrightarrow +i[H, \rho]$$

- Every population is unchanged: Rabi, decay, Ramsey and echo tests all pass
- Only phases flip: at $\Delta = 0$, $\langle \sigma_y \rangle(t) = \sin(\Omega t)$ becomes $-\sin(\Omega t)$

The bug your tests cannot see

A SIGN IN THE HAMILTONIAN TERM

$$-i[H, \rho] \longrightarrow +i[H, \rho]$$

- Every population is unchanged: Rabi, decay, Ramsey and echo tests all pass
- Only phases flip: at $\Delta = 0$, $\langle \sigma_y \rangle(t) = \sin(\Omega t)$ becomes $-\sin(\Omega t)$
- Your tests decide which bugs you can see: test every quantity you will publish

Rogues' gallery

WHAT GOES WRONG, AND THE DEFENCE

Failure	Defence
Invented or misquoted reference	Open it; check that it says that
Factor of 2, wrong sign	Analytic and limiting-case tests
Test weakened to pass	Commit tests first; read every test diff
Convention silently changed	Conventions in `AGENTS.md`, enforced by tests

Rogues' gallery

WHAT GOES WRONG, AND THE DEFENCE

Failure	Defence
Invented or misquoted reference	Open it; check that it says that
Factor of 2, wrong sign	Analytic and limiting-case tests
Test weakened to pass	Commit tests first; read every test diff
Convention silently changed	Conventions in `AGENTS.md`, enforced by tests

Rogues' gallery

WHAT GOES WRONG, AND THE DEFENCE

Failure	Defence
Invented or misquoted reference	Open it; check that it says that
Factor of 2, wrong sign	Analytic and limiting-case tests
Test weakened to pass	Commit tests first; read every test diff
Convention silently changed	Conventions in `AGENTS.md`, enforced by tests

Rogues' gallery

WHAT GOES WRONG, AND THE DEFENCE

Failure	Defence
Invented or misquoted reference	Open it; check that it says that
Factor of 2, wrong sign	Analytic and limiting-case tests
Test weakened to pass	Commit tests first; read every test diff
Convention silently changed	Conventions in `AGENTS.md`, enforced by tests

Rogues' gallery

QUIETER FAILURES

Failure	Defence
Invented or outdated function names	Run it; pin versions in `Manifest.toml`
Hz where rad/µs was meant	Units in names and docstrings; a test with a known period
Confident summary of a paper not read	Ask for the page and equation; check it
"All tests pass" that were never run	Run them yourself; CI runs them anyway

Rogues' gallery

QUIETER FAILURES

Failure	Defence
Invented or outdated function names	Run it; pin versions in `Manifest.toml`
Hz where rad/µs was meant	Units in names and docstrings; a test with a known period
Confident summary of a paper not read	Ask for the page and equation; check it
"All tests pass" that were never run	Run them yourself; CI runs them anyway

Rogues' gallery

QUIETER FAILURES

Failure	Defence
Invented or outdated function names	Run it; pin versions in `Manifest.toml`
Hz where rad/µs was meant	Units in names and docstrings; a test with a known period
Confident summary of a paper not read	Ask for the page and equation; check it
"All tests pass" that were never run	Run them yourself; CI runs them anyway

Rogues' gallery

QUIETER FAILURES

Failure	Defence
Invented or outdated function names	Run it; pin versions in `Manifest.toml`
Hz where rad/µs was meant	Units in names and docstrings; a test with a known period
Confident summary of a paper not read	Ask for the page and equation; check it
"All tests pass" that were never run	Run them yourself; CI runs them anyway

You are the author

- **Accountability**: the agent is not an author; every claim in the paper is yours

You are the author

- **Accountability**: the agent is not an author; every claim in the paper is yours
- **Data**: unpublished data, collaborators' code and personal data leave your machine when the agent reads them; check what your institution allows

You are the author

- **Accountability**: the agent is not an author; every claim in the paper is yours
- **Data**: unpublished data, collaborators' code and personal data leave your machine when the agent reads them; check what your institution allows
- **Disclosure**: journals ask how AI was used; know your journal's policy

You are the author

- **Accountability**: the agent is not an author; every claim in the paper is yours
- **Data**: unpublished data, collaborators' code and personal data leave your machine when the agent reads them; check what your institution allows
- **Disclosure**: journals ask how AI was used; know your journal's policy
- **Reproducibility**: record which tool and model helped, and when

Record what you used

```
$ git log -1
commit e4b1a20...
Author: Marcelo Janovitch
Date:   Tue Oct 20 14:02:11 2026 +0200

    Add Ramsey sequence with fringe-frequency test

    Assisted-by: <agent> (<model>)
```

Record what you used

```
$ git log -1
commit e4b1a20...
Author: Marcelo Janovitch
Date:   Tue Oct 20 14:02:11 2026 +0200

    Add Ramsey sequence with fringe-frequency test

    Assisted-by: <agent> (<model>)
```

- A trailer in the commit message: costs nothing, searchable later

Record what you used

```
$ git log -1
commit e4b1a20...
Author: Marcelo Janovitch
Date:   Tue Oct 20 14:02:11 2026 +0200

    Add Ramsey sequence with fringe-frequency test

    Assisted-by: <agent> (<model>)
```

- A trailer in the commit message: costs nothing, searchable later
- Some agents add one themselves (e.g. `Co-Authored-By`); keep it

Record what you used

```
$ git log -1
commit e4b1a20...
Author: Marcelo Janovitch
Date:   Tue Oct 20 14:02:11 2026 +0200

    Add Ramsey sequence with fringe-frequency test

    Assisted-by: <agent> (<model>)
```

- A trailer in the commit message: costs nothing, searchable later
- Some agents add one themselves (e.g. `Co-Authored-By`); keep it
- In the paper: one sentence in the acknowledgements or methods

Conclusion

- An LLM is a very large fit: plausible is not the same as correct
- An agent closes the loop, so it can check its work, if you give it checks
- Context is finite: one task per session, write the rest down
- Git is the safety net: commit often, read every diff
- Structure, pinned environments and CI: every result traceable and checked
- Verification is the job: tests that encode physics, written first, read by you

Backup: inside the transformer

ATTENTION

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d}}\right) V$$

- Q, K, V: queries, keys and values, linear maps of the token vectors
- d: dimension of the keys; softmax acts row by row
- Each token gathers information from all earlier tokens in the context
- Cost grows with the square of the context length

A. Vaswani et al. *Attention is all you need*, Advances in Neural Information Processing Systems **30** (2017)

Backup: colours in the schematics



- Anthracite: what the human does
- Petrol: what the model does, and the git history
- Turquoise: tools, files and side branches
- Basel red: only for what is new on a slide